Seller Product Catalog Page

Unified Requirements, Design & Data Specification

Generalised · Any Seller · Data-Driven · Fully Dynamic · Product-First

This document is the complete specification for building a standalone, independent Seller Product Catalog Page. It is fully generalised; no seller name or data is hard-coded. Pass this document together with a seller's Catalog JSON file to build their catalog page without any modifications to this spec. The catalog page is product-first — it prominently showcases the seller's actual products with cards, details, descriptions, specifications, and pricing, sources, ratings & reviews, socials etc. All data is read strictly and exclusively from the provided JSON; no data is scraped, crawled, or inferred from external sources.

1. Application Overview

Application Name: Seller Product Catalog Page

Description:

A standalone, independent dynamic, responsive, modern, and interactive web application serving as a unique product catalog page for each seller. The page aggregates all available catalog data from the provided JSON — company profile, products, media assets, social profiles, posts, and reviews — and renders only the sections for which data exists. The page is driven entirely by a per-seller Catalog JSON; it must never access, scrape, crawl, or infer any data from external URLs, APIs, or websites. All content displayed on the page — product names, descriptions, prices, images, contact info, social media posts, reviews — must be read exclusively from fields present in the supplied JSON object.

This is NOT affiliated with any marketplace platform. It is a fully standalone catalog page for the seller, intended to be built and hosted independently. All links, CTAs, and social references come exclusively from the seller's Catalog JSON — no platform URLs are constructed or assumed.

The page is a seller catalog page, not a generic profile page. Its primary purpose is to showcase the seller's products — with rich product cards, category browsing, and detailed product information — giving buyers a compelling, trust-building experience that drives enquiry and purchase intent.

UI/Design Reference:

For UI layout, design language, and component patterns, draw exclusively from the following reference websites — Gumroad, Lemon Squeezy (Lump), Pitch, Linear, Framer, Replit, Duolingo, CRED, Glossier, Dukaan, Squarespace, District Store. Study and replicate the animations, UI components, card styles, hover effects, and colour themes found only among these listed websites. The page must be very modern, attractive, catchy and highly interactive — every interaction should feel intentional and polished as seen on those reference sites.

Libraries to use as a reference:

https://glidejs.com/ , https://animate.style/, https://freefrontend.com/css-loaders/ , https://freefrontend.com/bootstrap-cards/ , https://threejs.org/ , https://spline.design/ (Spline preferred for 3D), GSAP ScrollTrigger, parallax effects, and https://michalsnik.github.io/aos/ (AOS — Animate On Scroll library — required for all scroll-triggered element animations throughout the page), particle js.

GENERALISATION RULE:

No seller name, address, product name, GSTIN, URL, phone, email, category, price, or any other seller-specific value may be hard-coded anywhere in the HTML/CSS/JS. Every piece of seller data

must be read from the uploaded Catalog JSON at runtime and embedded in HTML elements. This rule applies to ALL sections and ALL fields without exception.

DATA RULE:

All content on the page must come exclusively from the provided Catalog JSON file. The builder must NOT crawl any website, scrape any URL, call any external API, or infer/generate data on its own. If a field is absent or null in the JSON, the corresponding element or section is hidden — never populated with guessed or fabricated data.

2. Target Users & Use Cases

2.1 Target Users

- Buyers and visitors browsing seller product catalogs.
- Sellers whose product catalog pages are being rendered.
- Frontend developers or AI agents (e.g. Lovable, Claude Code) building the page from this spec and a Catalog JSON.

2.2 Core Use Cases

- Buyer visits a seller's unique catalog URL and sees their complete product catalog.
- Buyer browses product cards, reads descriptions and specifications, and makes an enquiry.
- Buyer filters or browses products by category.
- Buyer contacts the seller via WhatsApp, phone, email, or their own website.
- Buyer views the seller's social media posts (Instagram, YouTube) from social_profiles[].posts[].
- Buyer views reviews and ratings.
- Page dynamically adapts — only sections with available data are rendered.

3. Core Design Principles

- **Scalability & Flexibility** — Modular; easy to extend with new data sources or design elements
- **Fully Dynamic** — Every value from the Catalog JSON; nothing hard-coded or self-generated
- **Product-First** — Products are the centrepiece — cards, specs, imagery, pricing
- **Standalone & Independent** — No reliance on any third-party marketplace; direct seller-to-buyer
- **User-Centricity** — Seamless buyer experience; seller showcases products with zero effort
- **Brand Consistency** — Seller branding maintained throughout; modern, catchy, interactive
- **Performance & Responsiveness** — Mobile-first; fluid grids; all breakpoints from 320px to 1920px
- **Graceful Degradation** — Missing data = section hidden; no empty containers
- **Accessibility** — WCAG 2.1 AA; keyboard navigable; screen-reader friendly; touch targets ≥ 44×44px

4. Page Structure

Full tree of all sections. All sections are conditionally rendered based on data availability.

```
Seller Product Catalog Page (standalone, single page, no breadcrumb)

├── Sticky Navigation Bar                          [always shown, hamburger in mobile]
├── Hero Section (Seller Overview)                 [always shown]
├── About / Business Summary Section               [if company_profile fields available]
├── Contact & CTA Section (sidebar/sticky)         [always shown]
├── Product Catalog Section (Main)                 [if catalog_items[] or products[] non-
empty]
│   ├── Category Filter Bar                        [if 2+ distinct product_category
values]
│   ├── Product Card Grid (10–15 shown initially)  [always shown if products exist]
│   ├── "Show More Products" Toggle Button         [if total products > 15]
│   └── Product Detail Modal / Drawer              [on card click]
├── Media Gallery Section                          [if images or videos found]
│   ├── Images Tab (8–10 shown, lazy-load rest)
│   └── Videos Tab                                 [if YouTube posts available]
├── Social Posts Section                           [if social_profiles[].posts[].length >
0]
│   ├── Instagram Tab
│   └── YouTube Tab
│   └── LinkedIn, Twitter/X, Facebook etc.         [depending on post availability]
├── Reviews Section                                [if reviews_summary.no_of_ratings > 0]
│   ├── Aggregate Rating Card
│   └── Individual Review Cards                    [if reviews[] non-empty]
└── Footer + Mobile Sticky CTA Bar                 [always shown]
```

5. Section-by-Section Functional Specification

5.1 Sticky Navigation Bar

Set the HTML page <title> tag dynamically to the seller's name from json.company_profile.name so the browser tab shows the seller name. Implementation: document.title = json.company_profile.name; set this immediately after JSON is loaded, before any other rendering. Also update the <meta name="description"> tag with json.company_profile.description (or bio fallback, truncated to 160 chars). Additionally set og:title to the seller name and og:description to the description for social sharing previews. All meta values must come exclusively from the JSON; no hard-coded values. If company_profile.name is null/empty, fall back to a generic title(Seller Catalog). The title update must happen on every page load and reflect the loaded seller's name at runtime.

- Logo: Show seller logo (from social_profiles[instagram/youtube/facebook].profile_pic_url) OR styled text showing json.company_profile.name. Do NOT use any platform logo.
- Sticky nav anchor links: Overview | About | Catalog | Gallery | Social | Reviews&Ratings | Contact
- **Right side CTAs:** "Contact Seller" (outlined, links to WhatsApp or phone from json.company_profile.phones[0]) + "Get Quote" (filled accent). No marketplace CTA buttons.
- Hamburger menu on mobile
- Scroll-spy: active anchor updates as user scrolls through sections
- NO breadcrumb. No platform hierarchy. No external marketplace links.
- Highly interactive: hover effects, smooth transitions, 3D/Spline elements in background

Hamburger menu behaviour specification:

- Hamburger icon is HIDDEN in the hero section where nav background is transparent. Show hamburger ONLY in the sticky navigation bar (when user has scrolled and the nav is opaque/solid).
- Click hamburger → menu opens with slide-down or fade-in animation (200ms ease-in-out)
- Click any menu link → smooth-scroll to section + close menu

(1) Ensure ALL page sections have the correct id attributes: hero section → id="overview", about section id=”about”, product catalog → id="products", gallery → id="gallery", social posts → id="social", reviews → id="reviews", contact/CTA → id="contact". (2) Attach scroll listeners after DOM is fully loaded (DOMContentLoaded). (3) For the "Overview" link specifically: scrollTo top (scrollY=0) since it's the page top.Test on 375px mobile viewport: all 7 links must scroll to correct sections and close the menu.

- Click outside the menu → menu closes
- Scroll-spy updates active menu item while menu is open
- Test on 320px, 480px, 768px viewports

5.2 Hero Section

The hero section is the first impression. All data comes exclusively from the provided JSON. It must be visually rich and dynamic.

5.2.1 Hero Background / Banner

The hero banner background must be resolved using the following priority order:

1. **YouTube Channel Banner:** social_profiles[platform="youtube"].banner_url — Highest priority. Use as background-image.
2. **Facebook Cover Photo:** social_profiles[platform="facebook"].banner_url — Use if YouTube unavailable.
3. **When no banner URL is available (both youtube banner_url and facebook banner_url are null/missing), render a Particles.js animated canvas as the hero section background. Use particles.js (https://vincentgarreau.com/particles.js/) with a subtle, high-density particle configuration (small dots/lines, slow movement, light colours matching the page palette) so the hero section feels dynamic and alive even without a banner image. The particles canvas must be absolutely positioned behind all hero content (z-index: 0), and hero text/CTAs must remain on top (z-index: 1+). Disable particles on mobile (use CSS gradient fallback on max-width: 768px). Implementation: load particles.js from CDN, initialise with config only when hero_banner_url resolves to null after all fallback checks.**
4. **CSS gradient fallback:** Light-white gradient (see Appendix A for exact values). Final fallback.

For the hero banner image, use object-fit: contain.

UPDATED overlay — WCAG AA compliant for white text legibility:

```
.hero-overlay {
  background: linear-gradient(
    to bottom,
   rgba(0, 0, 0, 0.75) 0%,
   rgba(0, 0, 0, 0.65) 60%,
   rgba(0, 0, 0, 0.85) 100%
  );
  position: absolute; inset: 0; z-index: 1;
}
```

Other implementation requirements:

- Apply a parallax scroll effect to the banner on desktop only (see Section 6.5.2). Disable parallax on mobile (static background).
- Apply Ken Burns zoom animation to hero banner on desktop.
- If a banner URL fails at runtime (onerror), fall through to next source silently — never show empty banner.

5.2.2 Seller Logo / Avatar

Use real brand images from social_profiles. Priority order:

- **PRIMARY:** social_profiles[platform="instagram"].profile_pic_url
- **FALLBACK 1:** social_profiles[platform="youtube"].profile_pic_url
- **FALLBACK 2:** social_profiles[platform="facebook"].profile_pic_url
- **FALLBACK 3:** Initials circle from json.company_profile.name (max 2 letters), gradient background — use only if all above are null
- Display: circular avatar, white border, top-left of hero content. Subtle glow + hover effect(glowing light/glow shadow effect on hover).

5.2.3 Hero Text Content (all strictly from JSON)

Seller business name	json.company_profile.name
Business type / tagline	json.company_profile.business_type
Description / bio	json.company_profile.description (if non-null); else social_profiles[youtube].bio or social_profiles[instagram].bio
Website	json.company_profile.website — shown as "Visit Website" CTA if non-null
Rating	json.company_profile.rating_value — shown as star rating
Rating count	json.company_profile.rating_count — shown alongside rating
City	json.company_profile.city — direct field, no parsing required

Hero CTAs: "Visit Website" (if json.company_profile.website non-null) + "Contact Seller" (WhatsApp/phone). Remove marketplace-branded CTAs completely.

5.2.4 Trust Badge Strip (all from JSON, all clickable)

Render a horizontal pill badge strip. Each badge shown ONLY if data exists. All badges clickable:

Trust Badge	JSON Source	Click Behaviour
Rating: [N]★	json.company_profile.rating_value AND rating_count both non-null	Scrolls to Reviews section
[N] Reviews	json.review.length > 0	Scrolls to Reviews section
[City]	json.company_profile.city — direct field	Opens json.company_profile.google_location in new tab if non-null
[Business Type]	json.company_profile.business_type non-null	Scrolls to About section

[Y] Products shown	Showing Y (with data) Products	Scrolls to Product Catalog section
[N] Followers on the particular platform	Highest followers_count across all social_profiles[] entries, along with the platform name	Scrolls to Social Posts section
Visit Website	json.company_profile.website non-null	Opens website in new tab

Product Count Badge Logic(for reference)

The product count trust badge must ONLY show "Showing [N] products" where N = the number of products for which data is available(json.catalog_items.length). Always render as `Showing ${showcasedItems} products`.The tooltip (title attribute) may still show full detail for hover context. This applies to the trust badge strip in Section 5.2.4 and anywhere else the product count is shown as a badge.

5.2.5 Contact Person Info

- Phone: json.company_profile.phones[0] shown as inline pill link
- Email: json.company_profile.emails[0] shown as link
- Address: json.company_profile.address shown as subtle secondary line. LAYOUT FIX: Address display must handle long multi-part addresses without breaking layout. Implementation: render the address row as a flex container: display: flex; flex-wrap: wrap; gap: 4px; align-items: flex-start;. Each comma-separated part of the address can be a separate inline-flex span, or the full address string can be a flex item with flex: 1; min-width: 0; word-break: break-word; white-space: normal;. Do NOT use white-space: nowrap or overflow: hidden on the address element. The address row must accommodate up to 3 lines of text without clipping or horizontal overflow. Container width: 100% of its parent. Icon (map pin) is flex-shrink: 0 at the left. Text area beside it is flex: 1; min-width: 0 so it wraps naturally. This applies to all address display locations: hero contact pills (5.2.5), sidebar contact section (5.4), footer (5.10). CSS: .address-row { display: flex; gap: 6px; align-items: flex-start; width: 100%; } .address-icon { flex-shrink: 0; margin-top: 2px; } .address-text { flex: 1; min-width: 0; word-break: break-word; line-height: 1.4; }
- City: json.company_profile.city — direct string field

5.3 About / Business Summary Section

COMPANY_PROFILE.SOURCES[] AND DATA_SOURCES[] DISPLAY: The JSON contains two top-level source-of-truth fields on company_profile: (1) company_profile.sources[] — a flat array of all platforms from which ANY data about this seller was collected, e.g. [“website”, “gst”, “google”, “youtube”, “facebook”, “linkedin”]. (2) company_profile.data_sources[] — a simpler array of platforms actually scraped/fetched for content, e.g. [“website”, “youtube”]. DISPLAY: In the About / Business Summary section (5.3), render a “Verified across” or “Data sourced from” badge strip near the bottom of the section. Show one small pill per entry in company_profile.sources[] — each pill shows the platform icon + platform name (e.g. Google logo + “Google”). Label the strip: “Verified across [N] sources” where N = company_profile.sources[].length. Style: same small pill style as trust badge strip — inline-flex, border-radius 24px, padding 5px 12px, platform icon 12px, font-size 11px, muted border. This is purely informational — no click action. If sources[] is empty or absent, hide the strip. Also add a short tooltip on hover of each pill: e.g. “Business name verified via Google”.

Each item in the company_profile{} has array of objects like name, email, description etc. where there is a value and sources[] array, need to implement that source as well, for every cta, information or value used, there will be a small badge strip showing the "via source"

- **Hero Description Display:** Show company_profile.description with ellipsis (...) if text exceeds 150 characters. On ellipsis click, smooth scroll down to Section 5.3 (About/Business Summary) where the FULL description is displayed without truncation.
- **Social bio fallback:** social_profiles[youtube].bio → social_profiles[instagram].bio → social_profiles[facebook].bio
- Business type chip, Google Maps embed (json.company_profile.google_location iframe), website CTA button
- Social platform cards: for each social_profiles[] entry with non-null url — show icon, followers_count (formatted), bio (80 chars), Follow button
- DATA RULE: Show only fields present in JSON. If description and all bios are null, hide description area. Make sure to print the full description or make it ellipsis after 3 lines but on clicking (read more) it expands the description.

Section layout:

- Business type label shown as a prominent chip
- Google Maps embed: rendered using json.company_profile.google_location URL (iframe embed). If null, show address text only.
- Website link: json.company_profile.website as a prominent CTA button

For each entry in json.social_profiles[] where url is non-null, show a platform card in the sidebar with: platform icon, followers_count (formatted, e.g. "354K followers"), bio (truncated to 80 chars), and a "Follow" button linking to url. Show cards only for platforms with non-null url. Order by followers_count descending. NOTE: This sidebar social cards UI is UNCHANGED and must NOT be removed.

WHAT IS REMOVED: The "CONTACT PRESENCE" sub-card inside the full-width "Get in Touch" section. This card shows pill badges (Phone, Email, Website, WhatsApp) and appears at the bottom of the contact info column in the Get in Touch section. This card is REMOVED and replaced with a single "View Trust Graph" button. TRUST GRAPH MODAL SPEC: On clicking "View Trust Graph", a modal opens with ALL contact data in a tabular trust-graph format. MODAL LAYOUT (A) HEADERS: Col 1 = "Attribute". Cols 2+ = each source from company_profile.sources[] (e.g. "GST ✓", "Website ✓", "Google ✓", "Facebook ✓" etc). (B) DATA ROWS: Phones[N].value with Pri/Sec label, Emails[N].value, Bank a/c if available (masked **0876), Website URL, WhatsApp URL, Social URLs (Instagram/Facebook/YouTube/LinkedIn/Twitter), GST number, PAN. (C) CHECKMARKS: For each row, check sources[] array — show green ✓ (#4CAF50) if that source column is in sources[], else empty cell. (D) MODAL STYLE: white, max-width 700px, 80vh max, scrollable, close × top-right, sticky header row, alternating row shading, mobile = horizontal scroll. (F) DATA SOURCES: phones[], emails[], social_urls[], website, gst_number, pan_number from company_profile. (G) TOGGLE: Simple View | Detailed View. Detailed = table (default). Simple = vertical grouped list.

In the full-width "Get in Touch" contact section (the main page section showing WhatsApp / Call Now CTAs, phone list, email, address, and map), there is a "CONTACT PRESENCE" sub-card at the bottom. This card renders pills showing which contact channels are present (e.g. [Phone] [Email] [Website] [WhatsApp]). REMOVE this Contact Presence card entirely from the UI. In its place, render a single "View Trust Graph" button. This button opens the Trust Graph Modal (see Trust Graph Modal spec in the adjacent note). The social icon row in the same section (showing Instagram, YouTube, Facebook, LinkedIn icons from social_profiles[]) is KEPT AS-IS below this

button. NOTE: The right-sidebar social platform cards (showing follower counts, bios, Follow buttons) are a SEPARATE component and must NOT be removed — those stay unchanged. Only the Contact Presence pills card inside the main Get in Touch section is replaced.

5.4 Contact & CTA Section (Sidebar / Sticky)

SOURCE ATTRIBUTION ON CTA & CONTACT INFO: Every piece of contact information and every CTA button displayed in the Contact section (Section 5.4), the About section (Section 5.3), and any other location where company contact data is shown must display a small source attribution label indicating which platform/source the data was extracted from. Specification: (1) SOURCE TAG: Each contact item (phone number, email, WhatsApp, website URL) renders with a small "via [SourceName]" tag below or beside it. Example: a phone number extracted from the Google Business Profile shows "via Google"; one from the Instagram bio shows "via Instagram"; one from JustDial shows "via JustDial". The source tag is purely informational — it does NOT link or redirect anywhere. (2) JSON FIELD: Anticipate a source field on each contact item in the JSON. [JSON STRUCTURE CORRECTED FROM ACTUAL JSON] Actual JSON structure confirmed: company_profile.phones[N] = { value: "+91...", sources: ["website", "google", ...] } — field name is value (not number), and sources is a PLURAL ARRAY of strings (not a singular source string). Same for emails[N] = { value: "info@...", sources: ["website"] }. For the via [Source] tag: read sources[] array and join all values — e.g. sources: ["website", "google"] renders as "via Website, Google". If sources[] is absent or empty array, hide the source tag entirely. NOTE: This { value, sources[] } pattern applies universally to ALL company_profile fields (name, description, website, city, address, etc.) — every field is wrapped in this envelope. (3) SOURCE TAG STYLE: Small pill label, 10px font, muted background (var(--color-bg-section)), border 1px solid var(--color-border), colour var(--color-text-muted). Non-interactive. Example render: [+91 98XXXXXXXX] [via Google] on one row. (4) ABOUT SECTION SOCIAL PRESENCE: In Section 5.3 below the social profile cards, add a dedicated "Contact & Links" sub-section showing all available CTAs (WhatsApp, Call, Email, Visit Website) with their source tags. This mirrors the sidebar CTA section but embedded inline within the About section for users who reach it via scrolling. Each CTA row: icon + label + contact value + source tag pill. (5) This applies to ALL locations where phones[], emails[], or website is shown: hero section contact pills (5.2.5), sidebar CTA (5.4), about section (5.3), footer (5.10).

EMAIL & PHONE OWNERSHIP / LABEL DISPLAY: For every phone number and email address shown anywhere on the page, display a label indicating who it belongs to or what role/department it represents (e.g. Helpdesk, Customer Support, CEO, Sales Team, Owner, Accounts, Store, Head Office). Specification: (1) JSON FIELD: Anticipate a label or role field on each contact item. [JSON STRUCTURE CORRECTED] Actual JSON structure for phones: { value: "+91...", sources: ["website"] }. No label field currently exists in JSON. Anticipated future addition: { value: "...", sources: [...], label: "Customer Support" | "CEO" | "Helpdesk" | null }. Until label field is present, hide the ownership label entirely (do not show placeholder). Same pattern for emails[N] = { value: "...", sources: [...], label: "..." | null }. (2) DISPLAY: If label is non-null, show it prominently above or beside the contact number/email — rendered as a small bold tag, e.g. "Customer Support" in accent colour, or as a subtitle below the number. Format example: [Customer Support] +91 98XXXXXXXX [via Google]. (3) HIDDEN IF ABSENT: If label is null or field is absent from JSON, do not show any ownership label — no placeholder, no "Unknown", no empty space. The contact renders as before without the label. (4) APPLIES TO: Hero contact pills (5.2.5), sidebar CTA section (5.4), About section contact sub-section (5.3), footer (5.10), and any enquiry modal pre-fill. (5) MULTIPLE CONTACTS: When multiple phones or emails are shown, each renders with its own label independently. The label helps the buyer know which number to use for which purpose. (6) CTA BUTTON LABEL: The WhatsApp and Call

CTA buttons, when label is available, update their button text to “WhatsApp [Label]” or “Call [Label]” — e.g. “WhatsApp Support” or “Call Sales Team”. If label is null, button text stays as “WhatsApp” / “Call”.

The "Get in Touch" / contact CTA buttons in mobile view have insufficient padding and appear cramped. Fix: on mobile (max-width: 768px), all CTA buttons in the Contact section (WhatsApp, Call, Email) and the sticky mobile bottom bar must have: padding: 0.75rem 1.25rem minimum (vertical 0.75rem, horizontal 1.25rem). For the sticky bottom bar CTAs use: padding: 1px 1.5px so they feel full and tappable. Also apply: font-size: 1rem; font-weight: 600; border-radius: 0.8rem; min-height: 3rem; width: 100% (for full-width mobile CTAs). CSS: @media (max-width: 768px) { .contact-cta-btn, .sticky-cta-btn { padding: 1rem 1.5rem; min-height: 3rem; font-size: 1rem; font-weight: 600; border-radius: 10px; } .sticky-mobile-bar .btn { padding: 1rem 1.25rem;}}

When company_profile.phones[] has more than 1 entry, or company_profile.emails[] has more than 1 entry, do NOT show all numbers/emails at once in the CTA box. Instead use an accordion / progressive disclosure pattern: (1) INITIAL STATE: Show only the PRIMARY item — phones[0] as the main Call/WhatsApp CTA button, emails[0] as the main Email CTA. Each shows its source tag (“via Website”) and label if available. (2) VIEW MORE TRIGGER: Below the primary CTA(s), if there are additional phones or emails (N > 1), show a small text link: “+[N-1] more number(s)” or “+[N-1] more email(s)” — e.g. “+1 more number”, “+2 more emails”. Style: small text link (12px, accent colour, underline on hover), no button border. (3) ON CLICK: Expand to show all additional phone/email rows beneath the primary. Each additional item shows: formatted value (partial mask optional) + source tag + label tag (if available) + a secondary smaller CTA button (outlined, smaller than primary). Collapse again on a “Show less” link. (4) ANIMATION: Smooth expand/collapse with max-height transition (0 → auto via CSS transition trick: max-height: 0; overflow: hidden transitioning to max-height: 500px). (5) Desktop sidebar: same pattern but since there is more space, show up to 2 items before the accordion triggers (show first 2, “+N more” for the rest). (6) This applies to both the sidebar contact section (5.4) and the About section contact sub-section (5.3). (7) If phones[] has only 1 entry: no accordion needed, show normally. Same for emails[].

WhatsApp CTA (primary)	json.company_profile.phones[0] — wa.me link
Call CTA	json.company_profile.phones[0] — tel: link
Email CTA	json.company_profile.emails[0] — mailto: link
Address display	json.company_profile.address
City display	json.company_profile.city
Google Maps embed	json.company_profile.google_location (iframe embed) — hidden if null
Website CTA	json.company_profile.website — shown as link button if non-null
Social icon row	json.social_profiles[].url for each platform with non-null url

CTAs in this section are limited to: WhatsApp, Call, Email, Social icons. No marketplace platform links of any kind.

[JSON-CONFIRMED PARTIAL UPDATE] Social platform profile icons (Instagram, YouTube, Facebook, LinkedIn, Twitter) — source from json.social_profiles[].url where url is non-null. HOWEVER: WhatsApp CTA is an EXCEPTION — WhatsApp link is not in social_profiles[]; it comes from company_profile.social_urls[] where value starts with “api.whatsapp.com” or “wa.me”. Confirmed JSON structure: company_profile.social_urls[N] = { value: “https://api.whatsapp.com/send?phone=...”,

sources: [“website”, “shiva_enriched”] }. For WhatsApp button: detect the entry in social_urls[] whose value starts with api.whatsapp.com or wa.me and use that URL. social_urls[] also contains other social links (Facebook, LinkedIn, Twitter, Instagram, YouTube) as fallback — these should only be used if social_profiles[].url is null/absent for that platform. Priority: social_profiles[].url > social_urls[].value for non-WhatsApp platforms. For source attribution on WhatsApp CTA: read sources[] array from the matching social_urls[] entry (e.g. [“website”, “shiva_enriched”]).

5.5 Product Catalog Section (Primary Section)

This is the main section of the catalog page. It displays all products from the JSON in a rich, interactive grid of product cards. This section is always shown if json.catalog_items[] or json.products[] is non-empty.

Data source priority: prefer json.catalog_items[] over json.products[], as catalog_items contains enriched data including vision-confirmed alt text, specifications, and media URLs. Fall back to json.products[] if catalog_items is empty or absent.

Add a subtle particles.js canvas as a decorative background layer in the Product Catalog section. The canvas must be positioned absolute/fixed within the section container (z-index: 0) while all product cards, filter pills, and section content remain on top (z-index: 1+). Use a different, lighter config from the hero particles — for example: more particles, smaller size. Disable on mobile for performance(keep in hero section but not in product catalog). This same particles.js script (already loaded for hero) can be reused.

The particles in the Product Catalog section background must use a dark grey color scheme (NOT the light/white of the hero particles). Particle color: #4A4A4A to #6B6B6B range (dark grey). Connecting lines (if enabled): rgba(74, 74, 74, 0.3). Background of the canvas itself: transparent so the section background color shows through. Particle opacity: 0.35–0.5 (visibly dark grey but subtle enough not to interfere with card readability). The dark grey color distinguishes the catalog section particles from the hero section particles (which use light/page-palette colors) — this contrast is intentional. Config reference: { “particles”: { “color”: { “value”: “#555555” }, “line_linked”: { “color”: “#444444” }, “opacity”: { “value”: 0.4 }, “number”: { “value”: 60 }, “size”: { “value”: 2.5 } } }. Disable entirely on mobile (<= 768px) for performance.

5.5.1 Category Filter Bar

Shown only if there are 2 or more distinct product_category values across catalog_items.

- Extract unique values from catalog_items[].product_category
- Render as a horizontal scrollable pill/tab filter bar at the top of the section
- First pill: "All" — shows all products
- Remaining pills: each distinct product_category value
- On pill click: filter the product grid to show only cards matching that category (client-side JS filter)
- Active pill is visually highlighted with accent colour
- When a category is selected, apply the 10–15 card limit independently to that category's products.

For each product in catalog_items, source tiles show which platforms carry that product. CONFIRMED JSON: catalog_items[N].sources[] is a plain string array e.g. [“website”, “youtube”]. POSITION CHANGE: Source tiles are NO LONGER rendered below the tags/content area of the card. They are now an OVERLAY at the BOTTOM EDGE of the product card image — styled the same way as the “via Instagram” badge shown on social image cards.

Implementation: position: absolute; bottom: 0; left: 0; right: 0; inside the card image container (position: relative). TILE STYLE: dark semi-transparent pill: backdrop-filter: blur(4px); background: rgba(0,0,0,0.45); border: none; color: #fff; font-size: 10px; height: 22px; border-radius: 11px; padding: 2px 8px; icon 10px. Tiles are left-to-right horizontal. If tiles overflow the image width: the row is HORIZONTALLY SCROLLABLE (overflow-x: auto; scrollbar hidden; touch-scroll enabled) within the image bottom strip. Platform icon mapping: "website" →; □ globe; "youtube" →; ▶ red play; "instagram" →; camera; "facebook" →; f; "google" →; G; "linkedin" →; in. If sources[] empty or absent: no overlay. NO-IMAGE cards: show source tiles below product name (no image to overlay). If social_sources[] non-empty: show ▶ Watch thumbnail badge at bottom-right of image (separate from source tiles row). Real example: sources ["website", "youtube"] → [□ Website][▶ YouTube] pills at bottom of card image.

CATALOG_ITEMS[N].SOCIAL_SOURCES[] — PRODUCT-LINKED SOCIAL POSTS: Each catalog_items entry may have a social_sources[] array linking specific social media posts that directly feature or demonstrate that product.(social_sources[N] = { platform: "youtube", post_url: "https://www.youtube.com/watch?v=...", thumbnail_url: "https://i.ytimg.com/vi/.../hqdefault.jpg", caption: "Product installation/demo caption...", posted_at: "2020-06-03T05:55:56.000Z", likes: 7, views: 607 }). NOTE: post_url and thumbnail_url ARE populated with real YouTube data — implement fully. ON PRODUCT CARD: If social_sources[].length > 0, show a small video preview badge: a 40x40px thumbnail (rounded, thumbnail_url) with a play icon overlay, positioned at the bottom-left of the card image, or below the source tiles row. Clicking this thumbnail opens the product modal scrolled to the "See It In Action" section. If multiple social_sources, show up to 2 thumbnails overlapping (+N more label). ON PRODUCT MODAL: Add a "See It In Action" section below the product description. One card per social_sources[] entry: thumbnail (16:9, thumbnail_url), platform badge (YouTube icon top-left), caption (first 120 chars, truncated with ellipsis), likes + views row (♥ N likes, ▶ N views), "Watch on YouTube" button (opens post_url in new tab, accent outlined style). If social_sources[] is empty or absent, hide this entire section. Card width: 220px, horizontal scroll row if multiple.

5.5.2 Product Card Grid

DESKTOP LAYOUT: Use explicit CSS Grid. grid-template-columns: repeat(4, 1fr) at ≥1280px; repeat(3, 1fr) at 1024–1279px; repeat(2, 1fr) at 768–1023px; repeat(1, 1fr) at <768px.

Each product card displays the following fields, all sourced from catalog_items[N] (or products[N]):

Price text inside cards must remain visually restrained (no oversized font), stay within the card bounds, and never push outside the card container.

Initial page load: show ONLY 10–15 product cards. Keep remaining cards in the DOM but hidden (display:none or visibility:hidden). Reveal on toggle.

```
// Product card limit implementation
const INITIAL_LIMIT  = 12;  // configurable: 10–15
const allItems       = buildCatalogPage(json).items;
let   visibleItems   = allItems.slice(0, INITIAL_LIMIT);
let   isExpanded     = false;

function renderProductGrid(items) {
  const grid = document.querySelector(".product-grid");
  grid.innerHTML = "";
  items.forEach((item, index) => {
    const card = createProductCard(item);
    // AOS attributes (Change #5)
    card.setAttribute("data-aos", "fade-up");
    card.setAttribute("data-aos-delay", String((index % 4) * 100));
```

```
    card.setAttribute("data-aos-duration", "800");
    grid.appendChild(card);
    if (index >= INITIAL_LIMIT) card.classList.add("hidden-product");
  });
}

// Toggle button
showMoreBtn.addEventListener("click", () => {
  isExpanded = !isExpanded;
  document.querySelectorAll(".hidden-product").forEach(card => {
    card.style.display = isExpanded ? "" : "none";
  });
  showMoreBtn.textContent = isExpanded
    ? "Show Less"
    : `Show ${allItems.length - INITIAL_LIMIT} More Products`;
  if (isExpanded) AOS.refresh(); // re-trigger AOS for revealed cards
});

// Category filter: apply INITIAL_LIMIT per category
function filterByCategory(category) {
  const filtered = category === "all"
    ? allItems
    : allItems.filter(i => i.product category === category);
  renderProductGrid(filtered);
  updateToggleButton(filtered.length);
}
```

Each product card must have AOS data attributes for staggered scroll animation:

```
<!-- Product card with AOS -->
<div class="product-card"
     data-aos="fade-up"
     data-aos-delay="0"    /* 0ms, 100ms, 200ms... stagger per column position */
     data-aos-duration="800"
     data-aos-easing="ease-in-out"
     data-aos-offset="100"
     data-aos-once="true">
  <!-- card content -->
</div>
```

Card Element	JSON Field	Behaviour
Product image	catalog_items[N].primary_photo_url OR media_assets matched by fingerprint .url	object-fit:cover; onerror hides image; loading="lazy"
Alt text for image	catalog_items[N].alt_text OR media_assets matched .media_alt_text	Applied as img alt
Product name	catalog_items[N].clean_name	Bold headline, truncated at 2 lines
Category chip	catalog_items[N].product_category	Small pill badge on card
Sub-type chip	catalog_items[N].product_sub_type (if non-null)	Smaller secondary pill
Price	price_on_request=true → "Price on Request"; is_price_range=true →	Shown prominently; price_unit appended

	price_min–price_max; else price_single	
In-stock badge	catalog_items[N].in_stock	"In Stock" green / "Out of Stock" grey badge
Short description	catalog_items[N].description (100 chars max)	Secondary text colour
Tags	catalog_items[N].tags[] (first 3)	Small chip tags below description
Enquire CTA	Opens enquiry modal pre-filled with product name	Filled accent button

5.5.3 Product Detail Modal / Drawer

Clicking a product card (except the Enquire CTA) opens a detail modal:

When a product card that has no image (primary_photo_url is null or failed to load) is opened in the detail modal, do NOT show any "image not available" div, placeholder SVG, broken image icon, or empty image region. The modal must render only the content fields: product name, full description, specifications table, B2B attributes, price, tags, buyer persona, in-stock status, source URL, and AI visual description (if available). The image carousel area must be completely absent from the modal layout for no-image products — the modal should reflow to a single-column content-only layout. No empty space or placeholder where the image would normally appear. Implementation: check if photo_urls[] is empty/null before rendering the image section of the modal; if empty, skip the entire image block and render only the content block at full modal width.

Full product name	catalog_items[N].clean_name (or raw_name if different)
All images	catalog_items[N].photo_urls[] — mini image carousel
Full description	catalog_items[N].description — full text, not truncated
Specifications	catalog_items[N].specifications — key-value table (exclude null values)
B2B attributes	catalog_items[N].b2b_attributes — HSN, GST%, origin, brand, MOQ if non-null
Price	Same price logic as card
Tags	All tags from catalog_items[N].tags[]
Buyer persona	catalog_items[N].buyer_persona — "Ideal for: ..." in highlighted note box
In-stock status	catalog_items[N].in_stock
Source URL	catalog_items[N].source_url — "View Original Listing" link if non-null
AI visual description	If used_vision=true, append media_assets[fingerprint].visual_description

When a product detail modal is opened, show two distinct rows: (1) "Sourced From" ROW (non-clickable): Sourced from catalog_items[N].sources[] — a SIMPLE ARRAY OF STRINGS e.g. ["website", "youtube"]. Each string renders as a non-clickable pill (platform icon + name). This is purely informational — sources[] does NOT contain URLs. (2) "SEE IT IN ACTION" ROW

(clickable): Sourced from catalog_items[N].social_sources[] — see separate social_sources[] spec. These ARE clickable with actual post_url links. Old incorrectly-specced structure { platform, url, label } DOES NOT EXIST in actual JSON — remove that assumption. For backward compatibility: if source_url (string) is non-null on the item, show a single "View Original Listing" clickable tile using that URL. Position: "Available on" row sits below product title/price, before description. Style: non-clickable pills — 28px height, platform icon 14px, outlined border, muted text. REMOVE: the previous spec about tile.url and source.url — those fields do not exist in current JSON.

When building the source tiles and social links for a product in the modal, deduplicate URLs so the same URL never appears twice. Rule: (1) SOURCE PRIORITY: For social platform URLs, compare between social_profiles[].url (per-platform profile URLs) and company_profile.social_urls[] (flat URL list). Use whichever array has MORE entries (greater .length) as the richer dataset. If equal length, prefer social_profiles[]. (2) URL DEDUP within modal: after collecting all source entries from catalog_items[N].sources[] and resolving their company-level URLs, deduplicate by normalised URL string (lowercase domain, strip trailing slash, strip query string). If two sources resolve to the same URL, show only ONE tile. (3) Apply same dedup to social_sources[].post_url — if two entries share the same post URL, show only one. (4) Implementation: const uniqueSources = [...new Map(sources.map(s => [normaliseUrl(s.resolvedUrl), s])).values()]; where normaliseUrl strips trailing slash and query params. (5) The deduplication is computed once at modal-open time. This prevents repeated URLs appearing when both social_profiles[] and social_urls[] contain the same platform link.

5.5.4 Product Section CTA

- "Enquire About All Products" — opens general enquiry modal or links to WhatsApp with json.company_profile.phones[0]
- "View Seller Website" — shown only if json.company_profile.website is non-null. Links to that URL directly. Or link to product source url in JSON. No marketplace links.

5.5.5 Product Card Image Segregation

Separate product cards that have images from those that do not, using the following rules: (1) CARDS WITH IMAGES: Render all catalog_items[] entries where primary_photo_url is non-null and loads successfully — these are shown first in the product grid as normal product cards, with image displayed (object-fit: cover). (2) VISUAL SEPARATOR ROW: After all image-bearing cards, render a full-width horizontal divider row — a thin separator line with a subtle label such as "More Products" or "Also Available" in muted text (var(--color-text-muted)), indicating a new block begins below. (3) NO-IMAGE CARDS: Cards where primary_photo_url is null or fails to load (onerror) are NOT shown inline — instead, they are collapsed behind a single "Show [N] more products" summary card at the end of the grid. The summary card must show: icon (grid/stack icon), count of hidden no-image products (N), and label "more products available". Styled as a dashed-border card, same dimensions as a product card, with a subtle hover state. (4) ON CLICK of the "Show [N] more products" card: expand and render the no-image product cards directly below the separator, without full-page reload. No image placeholder div must be shown in these cards — render only product name, price, category chip, description, tags, and CTAs. (5) NEVER show an empty image placeholder, broken image icon, or "image not available" placeholder in the card grid. If an image fails to load, the card migrates to the no-image group silently via onerror handler. Implementation note: classify items on JSON load — items with primary_photo_url non-null go to withImageItems[], items with null go to noImageItems[]. Render withImageItems first, then separator, then the summary card for noImageItems.

5.6 Product Categories Section

NOTE:- This section has been REMOVED. Product categories are now displayed as filter chips within the Product Catalog section only. See Section 5.5 for the consolidated product catalog + category filter implementation.

5.7 Social Posts Section

This section shows the most recent social media posts from platforms that have post data in json.social_profiles[]. It is shown only if at least one social_profiles[] entry has posts[].length > 0.

5.7.1 Platform Tab Logic

- Iterate json.social_profiles[]. For each entry where posts[].length > 0, add a platform tab.
- Render a tab/toggle row at the top of the Social Posts section showing ALL of the following platforms regardless of post data availability: Instagram, YouTube, Facebook, LinkedIn, Twitter/X. Always show all 5 platform tabs. Platforms WITH posts data (posts[].length > 0): tab is fully active, clickable, shows post count badge. Platforms WITHOUT posts data (posts[].length === 0 or platform absent from social_profiles[]): tab is shown but DISABLED — greyed out, reduced opacity (0.45), “not available” or “—” indicator, not clickable, with a tooltip on hover: “No [Platform] posts available”. Tab order (fixed): Instagram | YouTube | Facebook | LinkedIn | Twitter/X. Each tab shows: platform icon + platform name + post count (or “—” if disabled). Style: active tab = accent colour fill; disabled tab = grey outline, muted text. The default active tab on load = the first platform that has post data. If ALL platforms have no posts, hide the entire Social Posts section as before.
- If only one platform has post data, show that platform's posts directly with no tab toggle visible.
- If zero platforms have post data (all posts[] are empty), hide the entire Social Posts section.

FACEBOOK, LINKEDIN, TWITTER/X POST DISPLAY: In addition to Instagram and YouTube (already specced in 5.7.2 and 5.7.3), add display support for Facebook, LinkedIn, and Twitter/X posts. For each platform, when its tab is active and posts[] data is available: FACEBOOK: Render post cards with thumbnail (if available), caption (120 chars), likes, comments, posted_at. Link “View on Facebook” opens social_profiles[platform=“facebook”].posts[N].post_url in new tab. No embed — use card format with thumbnail background. LINKEDIN: Render post cards with text content (120 chars), likes, comments, posted_at. No embed — card format. Link “View on LinkedIn”. TWITTER/X: Render tweet cards with tweet text (280 chars), likes, retweets, posted_at. Use Twitter/X oEmbed or card format with platform badge. Link “View on X”. All platforms follow the same profile header card spec (5.7.4) — show profile_pic_url, bio, followers_count, Follow button. JSON anticipated fields for all platforms: social_profiles[].posts[N] = { post_url, thumbnail_url, caption, likes, comments, views, posted_at }. If any field is null, hide that field from the card (do not show empty space).

5.7.2 Displaying Instagram Posts

Source: social_profiles[platform="instagram"].posts[]

PROBLEM: Instagram embed.js renders iframes at inconsistent heights based on post content type, breaking grid layouts.

SOLUTION: Force uniform container dimensions with overflow:hidden:

```
.instagram-embed-container {
```

Some Instagram posts return iframes with small or irregular dimensions (square, portrait, landscape posts have different native aspect ratios), causing embed cards to appear empty, too

short, or inconsistent in the social posts grid. Fix: (1) Set a FIXED MINIMUM HEIGHT on all instagram embed containers: min-height: 480px; this prevents collapse for small-dimension posts. (2) Force uniform card height in the grid using a CSS wrapper: .instagram-post-card { height: 540px; overflow: hidden; position: relative; }. (3) The instagram-media iframe / blockquote inside must fill the container: .instagram-media { min-width: 100% !important; width: 100% !important; min-height: 480px !important; margin: 0 !important; }. (4) DO NOT use instagram oEmbed script for sizing — override it with !important CSS after the embed.js runs: use a MutationObserver to detect when the iframe is inserted and then apply: iframe.style.minHeight = "480px"; iframe.style.height = "480px";. (5) For posts that still fail to load (embed.js returns error or iframe is blank): detect via iframe load event — if contentDocument is empty or height < 100, show a fallback card: post thumbnail (from social_profiles[].posts[N].thumbnail_url), caption text, and a "View on Instagram" link. (6) ALL instagram post cards in the grid must have the same rendered height — use CSS grid with grid-auto-rows: 540px or align-items: stretch on the posts container. This ensures no gaps or irregular card heights regardless of post dimension.

```
  width: 100%;
  max-width: 400px;
  height: 550px;             /* fixed height — adjust between 500–600px */
  overflow: hidden;
  border-radius: 12px;
  position: relative;
  display: flex;
  align-items: flex-start;
  justify-content: center;
  background: var(--color-bg-card);
}

.instagram-embed-container iframe {
  width: 100% !important;
  height: 600px !important;  /* slightly taller than container; hidden by overflow */
  min-width: unset !important;
  border: none;
}

/* Fallback placeholder card (shown while embed.js loads) */
.ig-placeholder {
  width: 100%; height: 100%;
  background-image: var(--ig-thumb-url);  /* thumbnail_url as CSS var */
  background-size: cover; background-position: center;
  display: flex; flex-direction: column;
  justify-content: flex-end; padding: 16px;
  border-radius: 12px;
}

/* If embed.js still doesn't fit — apply rescale */
.instagram-embed-container .instagram-media {
  transform: scale(0.95);
  transform-origin: top center;
}
```

Post Field	JSON Path	Usage
Post type	social_profiles.find((profile) => profile.platform === "instagram").posts[n].type—("post" or "video")	Used to determine embed strategy

Post URL	social_profiles.find((profile) => profile.platform === "instagram").posts[n].post_url	Primary embed source — use as Instagram blockquote permalink
Thumbnail URL	social_profiles.find((profile) => profile.platform === "instagram").posts[n].thumbnail_url	Secondary CDN image — use as placeholder card background while embed loads. Apply onerror to hide if expired.
Caption	social_profiles.find((profile) => profile.platform === "instagram").posts[N].caption	Truncate to 120 chars with "..." for card display. Show full caption in expanded view.
Likes	social_profiles.find((profile) => profile.platform === "instagram").posts[N].likes	Show on card as "♥N likes"
Comments	social_profiles.find((profile) => profile.platform === "instagram").posts[N].comments	Show on card as "□ N comments"
Views	social_profiles.find((profile) => profile.platform === "instagram").posts[N].views (null for non-video)	Show only if non-null as "▸ N views"
Posted at	social_profiles.find((profile) => profile.platform === "instagram").posts[N].posted_at (ISO 8601)	Format as relative time (e.g. "2 days ago") or short date

Rendering strategy for ALL Instagram post types (both "post" and "video"):

- Render as Instagram blockquote embed using post_url as the permalink:

Always use the exact post_url as the permalink source for Instagram embeds; do not substitute any other embed URL.

```
<blockquote class="instagram-media"
  data-instgrm-permalink="{ social_profiles.find((profile) => profile.platform === "instagram").posts[N].post_url}"
  data-instgrm-version="14">
</blockquote>
<script async src="//www.instagram.com/embed.js"></script>
```

- Load embed.js via IntersectionObserver (lazy — only when section enters the viewport).
- While embed.js is loading, show a styled placeholder card using: thumbnail_url as background, caption (120 chars), likes, comments, views (if non-null), posted_at, and Instagram logo watermark. All from JSON fields — no image dependency.
- If embed.js fails to load within 5 seconds, keep the placeholder card permanently. Do NOT attempt CDN image as fallback.
- Sort posts by posted_at descending. Show up to 5 most recent posts.

Show a "Follow on Instagram" button linking to json.social_profiles[platform="instagram"].url.

5.7.3 Displaying YouTube Posts/Videos

Source: social_profiles[platform="youtube"].posts[]

Post Field	JSON Path	Usage
Post URL	social_profiles.find((profile) => profile.platform === "youtube").posts[N].post_url	Contains full YouTube URL. Extract video ID for embed.
Thumbnail URL	social_profiles.find((profile) => profile.platform === "youtube").posts[N].thumbnail_url	Use directly as card thumbnail image. Stable ytimg.com URL.
Caption / Description	social_profiles.find((profile) => profile.platform === "youtube").posts[N].caption	Truncate to 120 chars for card. Full text in modal.
Likes	social_profiles.find((profile) => profile.platform === "youtube").posts[N].likes	Show on card
Comments	social_profiles.find((profile) => profile.platform === "youtube").posts[N].comments	Show on card
Views	social_profiles.find((profile) => profile.platform === "youtube").posts[N].views	Show prominently on card (YouTube videos always have views)
Posted at	social_profiles.find((profile) => profile.platform === "youtube").posts[N].posted_at (ISO 8601)	Format as relative time or short date

Video ID extraction and embed:

```
function extractYouTubeId(url) {
  // Handles: watch?v=, shorts/, youtu.be/
  const watchMatch = url.match(/[?&]v=([^&]+)/);
  if (watchMatch) return watchMatch[1];
  const shortsMatch = url.match(/shorts\/([^?&]+)/);
  if (shortsMatch) return shortsMatch[1];
  const shortMatch = url.match(/youtu\.be\/([^?&]+)/);
  if (shortMatch) return shortMatch[1];
  return null;
}

// Embed URL:
const videoId = extractYouTubeId(post.post_url);
const embedUrl = `https://www.youtube.com/embed/${videoId}`;
// Show in modal iframe on card click
```

- Sort posts by posted_at descending. Show up to 5 most recent videos.
- Each video card: thumbnail_url as background image, play button overlay, views + likes + posted_at in a footer strip, caption truncated to 120 chars.
- Clicking card opens an iframe YouTube embed in a modal. Use the embed URL constructed from extracted video ID.
- Show a "Subscribe" button linking to json.social_profiles[platform="youtube"].url.

5.7.4 Platform Profile Header

At the top of each platform's post section, show a profile header card using data from social_profiles[]:

- Profile picture: social_profiles[platform].profile_pic_url (if non-null) — circular avatar
- Bio: social_profiles[platform].bio (truncated to 100 chars)
- Followers count: social_profiles[platform].followers_count (formatted: 354K, 1.2M)
- Follow button: links to social_profiles[platform].url
- If profile_pic_url, bio, or followers_count is null, hide that element (do not show empty space)

5.8 Media Gallery Section

Show images and videos collected from the JSON. The section is hidden entirely if no images and no videos are found.

IMPORTANT: All images must come from the JSON only. Do not fetch or crawl any external URLs beyond what is directly stored in JSON fields.

5.8.1 Image Sources — UPDATED PRIORITY ORDER

1st Priority (NEW)	Social media post thumbnails: collect all unique thumbnail_url values from social_profiles[].posts[].thumbnail_url. These are shown first in the gallery.
2nd Priority (NEW)	media_assets[N].url where scene_type = "product_image" or media_confidence ≥ 0.8
3rd Priority	catalog_items[N].photo_urls[] — all unique URLs across catalog items
4th Priority	catalog_items[N].primary_photo_url — ensures no omission
Deduplication	Same URL = same image; deduplicate across ALL sources before rendering. STRENGTHENED DEDUP RULE: (1) URL normalisation before comparison: lowercase the full URL, strip trailing slash, strip query string parameters (everything after ?), strip fragment (#). Two URLs that are the same after normalisation are considered duplicates — keep only the first occurrence. (2) Dedup must apply across ALL image pools — social thumbnail URLs, media_assets[] URLs, and catalog_items[].photo_urls[] — as a single unified set. (3) Also deduplicate within each pool independently before merging. (4) Implementation: after building the full gallery_images[] array, run: const seen = new Set(); const deduped = gallery_images.filter(img => { const key = normaliseUrl(img.url); if (seen.has(key)) return false; seen.add(key); return true; }); (5) If an image URL fails to load (onerror), remove it from the gallery silently — do not show a broken image placeholder in the gallery grid.

```
// Build ordered image list (social first, then catalog)
function buildGalleryImages(json) {
  const social_thumbs = (json.social_profiles || [])
    .flatMap(sp => (sp.posts || [])
      .filter(p => p.thumbnail_url)
```

```
      .map(p => ({ url: p.thumbnail_url, badge: "Social", alt: p.caption?.slice(0,80) ||
"" })));

  const media_imgs = (json.media_assets || [])
    .filter(m => m.media_confidence >= 0.8)
    .map(m => ({ url: m.url, badge: "Product", alt: m.media_alt_text, caption:
m.visual_description }));

  const catalog_imgs = (json.catalog_items || [])
    .flatMap(i => (i.photo_urls || []).map(u => ({ url: u, badge: "Gallery", alt:
i.alt_text || "" })));

  const all = [...social_thumbs, ...media_imgs, ...catalog_imgs];
  return all.filter((v,i,a) => a.findIndex(x=>x.url===v.url)===i); // deduplicate
}

// Show 8-10 initially; lazy-load rest
const GALLERY_INITIAL = 9;
const allImages   = buildGalleryImages(json);
const initialImgs = allImages.slice(0, GALLERY_INITIAL);
const lazyImgs    = allImages.slice(GALLERY_INITIAL);

// Render initial 9 eagerly
initialImgs.forEach(img => renderGalleryTile(img, false));

// Lazy-load rest via IntersectionObserver
const sentinel = document.querySelector(".gallery-sentinel");
const obs = new IntersectionObserver((entries) => {
  if (entries[0].isIntersecting && lazyImgs.length > 0) {
    const batch = lazyImgs.splice(0, 6);
    batch.forEach(img => renderGalleryTile(img, true)); // loading="lazy"
    if (lazyImgs.length === 0) obs.disconnect();
  }
}, { rootMargin: "200px" });
obs.observe(sentinel);
```

Image Display Rules

- CSS grid: 3 cols desktop / 2 cols tablet / 1 col mobile (enforced with explicit grid-template-columns)
- Click opens a lightbox modal showing the full image with caption and visual description from media_assets
- Each image card has a small source badge ("Product Image", "Gallery")
- Apply loading="lazy" and an onerror handler that hides the tile silently if the URL fails to load
- Broken image URLs: hide the tile from DOM silently using onerror handler — do not show broken image icon or placeholder SVG

5.8.2 Video Sources

Videos now come from json. social_profiles.find((profile) => profile.platform === "youtube").posts[] in addition to any YouTube URLs in social_urls[].

Detection	Source	Embed Strategy

YouTube video/shorts	social_profiles.find((profile) => profile.platform === "youtube").posts[N].post_url	Use thumbnail_url as card thumbnail; iframe embed on click
URL contains youtube.com/watch?v= or /shorts/	json.social_urls[] (fallback)	Extract video ID, iframe embed
URL ends with .mp4, .webm	Any JSON field	HTML5 <video> player in popup modal

If no video URLs are found in the JSON, the Videos tab is hidden. If no images either, the entire Media Gallery section is hidden.

5.9 Reviews Section

5.9.1 Aggregate Rating Card

Overall rating	json.reviews_summary.total_rating — large numeric with visual stars
Rating count	json.reviews_summary.no_of_ratings — "Based on N ratings"
Source badge	"Google" (sourced from Google Maps data in company_profile)
Consistency check	json.company_profile.rating_value / json.company_profile.rating_count — should match reviews_summary

5.9.2 Individual Review Cards

- **Source: json.reviews[] — IT MIGHT CONTAIN INDIVIDUAL REVIEWS with author, rating, text, date, is_local_guide, owner_response; show only aggregate card if empty**
- DO NOT use reviews_summary.rating_comments[] to populate Customer Voices review cards. These are the same reviews as reviews[] and cause duplication. Individual review cards in Customer Voices must ONLY come from the json.reviews[] array.
- Each card: reviewer name (if available), visual star rating, review text, source badge (platform name, e.g. "Google", "JustDial", "Facebook"), date (if available)

MULTI-SOURCE RATINGS & REVIEWS: The reviews section must support and display ratings & reviews from multiple sources beyond Google, including but not limited to: JustDial, Facebook Business/Marketplace, IndiaMart, TradeIndia, and any other platform from which review data may be present in the JSON. UI implementation (build UI even if JSON data not yet present for non-Google sources): (1) SOURCE TAB STRIP: Render a horizontal tab strip at the top of the Reviews section showing all review sources found in the data. Tabs: “All”, “Google”, “JustDial”, “Facebook” etc. Each tab shows the platform logo/icon + platform name + total review count for that platform in parentheses. The “All” tab aggregates all reviews from all sources. Active tab is accent-highlighted. (2) SOURCE BADGE ON EACH CARD: Every individual review card must display a small coloured source badge pill (top-right corner of card) showing which platform the review came from — e.g. a Google “G” coloured badge, JustDial orange badge, Facebook blue badge. Badge colour matches the platform brand colour. (3) AGGREGATE RATING CARD: Show separate aggregate ratings per source if available — e.g. “Google: 4.5★ (120)”, “JustDial: 4.2★ (45)” as stacked rows. If only one source is present, show

single aggregate as before. (4) JSON STRUCTURE (anticipated): reviews[] items will have a source field (e.g. source: "google" | "justdial" | "facebook") and reviews_summary will have per-source aggregates. Until that data is present in JSON, the UI renders only Google data with the tab strip pre-built for multi-source. (5) EMPTY STATE: If a platform tab has 0 reviews, show an empty state within that tab: "No [Platform] reviews available yet" — do NOT hide the tab if the platform is expected (e.g. JustDial tab can be shown disabled/greyed if 0 reviews). (6) Graceful fallback: reviews[] items might have NO source field in the JSON (confirm it from actual JSON inspection). All current reviews come from Google Maps via company_profile.reviews_api_url (Google Maps Reviews API). Therefore: treat ALL current reviews as source="google" by default. The multi-source tab UI (All / Google / JustDial / Facebook) must be built and ready; only the Google tab will have data until other sources are added to JSON. The source field on reviews[N] is a FUTURE addition — when added, each review will carry source: "google" | "justdial" | "facebook" etc. Current reviews[] fields confirmed: { author, rating, text, original_lang, date, likes, is_local_guide, owner_response }.

- If reviews[] is empty or absent: show aggregate card only with note "Individual reviews not available". [CHANGE 6: DO NOT fall back to rating_comments[] — it is the same data and causes duplication]

5.9.3 Review Section Layout

- Show aggregate rating card prominently at the top with large star display

Show "via Google" source tag on the aggregate rating when company_profile.rating_value.sources[] = ["google"].

On desktop, when individual review cards are displayed in the adjacent "Customer Voices" column, the Aggregate Rating Card (left column) has visible empty/white space below the rating content. Fix: (1) Use CSS flexbox stretch on the reviews section row so the rating card fills the full height of the row: .reviews-row { display: flex; align-items: stretch; } .rating-card { display: flex; flex-direction: column; justify-content: space-between; height: 100%; }. (2) Add a decorative bottom filler to the rating card: a large background watermark star icon (opacity: 0.05–0.08), or a section with review distribution bar chart (count per star 1–5, derived from reviews[] array if available). (3) If distribution data is not available, fill the empty space with a visually styled quote block or a "What customers say" summary text. The rating card must never have large blank areas when reviews are displayed beside it on desktop (>768px).

Individual Reviews Display: Each review from json.reviews[] array, must render as a card showing: (1) Author name, (2) Star rating (visual 1-5 stars), (3) Review text (truncated with 'Read more' if >200 chars), (4) Date posted, (5) 'Local Guide' badge if is_local_guide=true, (6) Owner response (if owner_response non-null). Grid: 2-column desktop, 1-column mobile.

- Individual review cards below in a 2-column grid on desktop, 1-column on mobile
- Do NOT compute a blended average from multiple sources — show only the data as it exists in the JSON

Star icons must be visually coloured (#F59E0B) and clearly visible against the page background

Replace the previously present empty/placeholder div in the reviews section (formerly reviewsection.tsx line 110 area — the large container with small text that had no functional purpose) with a Star Filter Strip. The star filter strip sits between the Aggregate Rating Card and the Customer Voices card. Specification: (1) Render a horizontal strip with 5 filter buttons, one per star rating: "★★★★★ 5 Stars", "★★★★ 4 Stars", "★★★ 3 Stars", "★★ 2 Stars", "★ 1 Star", plus an "All" pill at the start. (2) Each pill shows its count of reviews with that rating in parentheses, e.g. "5 Stars (12)" — derived by counting reviews[].rating === N from the JSON. (3) On clicking a star pill, filter the Customer Voices (individual review cards) to show only reviews matching that star rating. The Aggregate Rating Card is unaffected by the filter. (4) Active pill:

highlighted with accent color (var(--color-accent)) and filled background; inactive pills: outlined/ghost style. (5) If a star rating has 0 reviews, show the pill in a disabled/greyed state (not clickable). (6) "All" pill resets the filter to show all reviews. (7) The filter is client-side JS only — no network request. (8) Animate the review card transition on filter change: fade-out old cards (150ms), fade-in new filtered cards (200ms). Desktop layout: strip renders horizontally between the two columns. Mobile: strip renders as a horizontal scrollable row above the review cards.

The Customer Voices box (the scrollable container that shows individual review cards) must implement slow auto-scroll when the content overflows the visible area. Specification: (1) Auto-scroll direction: vertical, downward, continuous loop. Scroll speed: slow — 0.5px per animation frame (approximately 30px/second at 60fps). (2) The auto-scroll must PAUSE immediately when the user clicks anywhere inside the Customer Voices container, or when the user manually begins scrolling (touchstart / mousedown). (3) On mouseenter / touchstart: pause auto-scroll and allow full manual control. On mouseleave / touchend (after 2.5 seconds idle): resume auto-scroll from current position. (4) The scroll loops: when the container reaches the bottom, it smoothly resets to top and continues (seamless loop — duplicate content at bottom for seamless effect if needed, or use requestAnimationFrame with scrollTop reset). (5) Implementation: use requestAnimationFrame loop; track a paused flag toggled by user interaction events. Do NOT use CSS scroll-behavior or CSS animations for this — use JS RAF loop for precise control. (6) If there are 3 or fewer review cards (content does not overflow), do NOT enable auto-scroll — leave as static content. (7) Show a subtle scrollbar (custom styled, thin, 3px, semi-transparent) when auto-scrolling so users know the content is scrollable.

5.10 Footer & Sticky Mobile CTA

- Sticky bottom bar on mobile: WhatsApp + Call buttons full width from json.company_profile.phones[0]
- Footer col 1: Seller logo (social_profiles profile_pic or initials) + business name + business type
- Footer col 2: Quick links — Overview | Products | Categories | Gallery | Social | Reviews | Contact (no marketplace link)

If The "Overview" quick link in Footer col 2 is not scrolling to the hero/overview section. Fix: ensure the hero section has id="overview" (or id="hero") set on the element. The footer anchor must be href="#overview" and use smooth scroll. Add explicit JavaScript: document.querySelector('a[href="#overview"]').addEventListener('click', (e) => { e.preventDefault(); window.scrollTo({ top: 0, behavior: 'smooth' }); }); — OR — ensure the hero section top element has the correct matching id. This fix applies to BOTH the footer quick links AND the sticky nav "Overview" link. Verify all nav anchor IDs: #overview → hero top, #products → product catalog section, #categories → categories section, #gallery → media gallery, #social → social posts, #reviews → reviews section, #contact → contact/CTA section.

- Footer col 3: Social icons from json.social_profiles[].url (non-null entries only) + email from json.company_profile.emails[0]
- Footer bottom: Copyright year only. No third-party platform branding.

6. UI/UX Design Reference

Draw exclusively from: Gumroad, Lemon Squeezy, Pitch, Linear, Framer, Replit, Duolingo, CRED, Glossier, Dukaan, Squarespace, District Store.

Libraries: https://glidejs.com/, https://animate.style/, https://freefrontend.com/css-loaders/, https://freefrontend.com/bootstrap-cards/, https://threejs.org/, https://spline.design/, https://michalsnik.github.io/aos/ (AOS), GSAP ScrollTrigger, parallax.

6.1 Colour Palette

Use curated light-white palette colours (see Appendix A for full palette recommendations and hex codes). Light theme throughout — no dark backgrounds on primary sections.

Keep the theme light, but introduce richer accent colour moments, gentle gradients, and subtle animated highlights so the page feels more alive and less washed out.

Colour Role	Updated Value (v7)	Usage
Page Background	#F5F1E8 (soft cream) or #FAF9F6 (warm white)	Primary page background — NOT plain #FFFFFF
Section Alt Background	#EDE8DC (warm beige) or #F0EDE8	Alternating section backgrounds
Card Background	#FFFFFF or #FDFCFA	Cards use slightly warmer white; shadow: 0 1px 3px rgba(0,0,0,0.06)
Text Primary	#2C2118 (deep warm charcoal)	Body text, headings — NOT pure #000000 or #111111
Text Secondary	#6B5E52 (warm brown-grey)	Meta, descriptions, timestamps
Primary Accent	#1A1A1A or #2563EB	Headlines, primary CTA buttons
Secondary Accent	#10B981 (green)	WhatsApp, in-stock badges
Gradient (hero mobile)	135deg, #F5F1E8 → #EDE8DC	Mobile hero gradient (Change #7)
Star Rating	#F59E0B	Review stars
Social Post Card	Platform brand (IG: #E1306C, YT: #FF0000, FB: #1877F2)	Platform header strip accent

CSS Custom Properties — define these at :root level. All components reference these variables (never hardcode colours):

```
:root {
  /* CHANGE #12 — Light-white palette variables */
  --color-bg-primary:    #F5F1E8;  /* soft cream — main page bg */
  --color-bg-section:    #EDE8DC;  /* warm beige — alternate sections */
  --color-bg-card:       #FFFFFF;  /* card background */
  --color-bg-hero-mob:   linear-gradient(135deg, #F5F1E8 0%, #EDE8DC 50%, #F5F1E8 100%);
  --color-text-primary:  #2C2118;  /* deep warm charcoal */
  --color-text-secondary:#6B5E52;  /* warm brown-grey */
  --color-text-muted:    #9C8F85;  /* muted warm grey */
  --color-accent:        #2563EB;  /* primary CTA accent */
  --color-accent-green:  #10B981;  /* WhatsApp / in-stock */
  --color-border:        #DDD7CE;  /* warm light border */
  --color-shadow:        rgba(44, 33, 24, 0.08);
}
```

6.2 Typography

- Font stack: "Inter", "DM Sans", "Segoe UI", Arial, sans-serif — import Inter and DM Sans from Google Fonts
- Hero seller name: clamp(24px,4vw,48px), font-weight: 800, letter-spacing: -0.02em, color: var(--color-text-primary) or white (based on hero bg)
- Product card name: clamp(14px,1.5vw,18px), font-weight: 700
- Section headings: clamp(18px,2.5vw,28px), font-weight: 700
- Body text: 15px, font-weight: 400, line-height: 1.7, color: var(--color-text-primary)
- Price text: 18–22px, font-weight: 700
- Meta / secondary: 13px, color: var(--color-text-secondary)
- Labels / badges: 11–12px, font-weight: 600, letter-spacing: 0.05em, uppercase

6.3 Responsive Breakpoints

Breakpoint	Viewport	Layout Changes
Mobile (XS)	320–359px	Single column everything; product cards full-width; hero vertical stack; font-size reduced; category tiles single column; touch targets 44px min
Mobile (Small)	360–479px	Single column; font-size reduced; 1 col products; 1 col gallery; hamburger nav
Mobile (Large)	480–767px	1 col products; 2 col gallery; sticky CTA bar; hamburger nav
Tablet (Portrait)	768–899px	Full width; 2 col products; 2 col gallery; mobile CTA bar
Tablet (Landscape)	900–1023px	Full width, no sidebar; 3 col products; 3 col gallery
Desktop (Standard)	1024–1279px	Main content + 290px sidebar; 3 col products; 3 col gallery
Desktop (Large)	≥1280px	Main content (flex:1) + 320px sidebar; 4 col products; 4 col categories; 3 col gallery

6.4 Component Patterns

All these patterns MUST be used as a foundation; builders are encouraged to be creative, modern, and catchy. Apply glassmorphism, border effects, gradients, hover animations, Three.js, parallax effects, glow on hover for buttons etc.

Use a lighter but more expressive visual system overall: soft color pops, animated gradients, hover lifts, glowing badges, and polished micro-interactions across hero, cards, chips, and CTAs.
Missing Product Image Handling: If product.primary_photo_url is null or empty: (1) Display a light grey placeholder div (same height/aspect as normal cards - 1:1 or 4:3). (2) Show icon in center: □ (camera emoji) or custom icon. (3) Text below icon: 'No image available'. (4) Expand product title + description area to fill the image space. (5) Keep card readable and don't collapse.

- Product Cards: White/cream/off-white bg, border-radius: 12px, 1px solid #E5E7EB border, layered box-shadow. Hover: lift + scale + shadow. Price bold. In-stock badge top-right. Keep

product-card prices compact and contained: use clamp sizing, no oversized price text, no overflow outside the card, and let longer values wrap or truncate cleanly.

- Social Post Cards: Platform header strip with colour accent and icon. Thumbnail image (fixed aspect ratio 1:1). Caption (2 lines max). Engagement stats row (likes, comments, views). Timestamp footer. All from JSON fields.
- Social Profile Header Card: Circular profile_pic_url avatar, platform name + icon, followers_count formatted (354K), bio truncated, Follow CTA button.
- Category Tiles: Creative layout (bento/masonry). 3D tilt on hover, glow border, glassmorphism. Image top, count label below.
- Filter Pills: Rounded pill buttons; active state accent colour; smooth 0.2s transition.
- Buttons: Gradient or solid fill; smooth hover; border-radius: 8px; visible focus ring.
- Skeleton loaders: Shimmer animation: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%).

The two-column modal layout with sticky left image IS PRESERVED. This is intentional UX. The fix targets small-height screens where right content gets stuck. FULL SPEC: (1) MODAL CONTAINER: position: fixed; top: 50%; left: 50%; transform: translate(-50%, -50%); max-height: 90vh; overflow: hidden; — the outer modal does NOT scroll. (2) LEFT COLUMN (image area): width: 45%; position: sticky; top: 0; height: 100%; max-height: calc(90vh - 48px); overflow: hidden. Image carousel: height: 100%; object-fit: contain. The image stays locked in place as right side scrolls. (3) RIGHT COLUMN (content area): width: 55%; height: calc(90vh - 48px); overflow-y: auto; — ONLY this column scrolls. All specs (description, tags, CTAs, social links etc.) inside scroll naturally. (4) ROOT CAUSE FIX for "stuck halfway" on Windows laptops (low height screens): ensure NO child inside right column has: position: absolute with height exceeding the column; or overflow: visible causing content to escape scroll container; or flex/grid layout that grows beyond the column height without scrolling. Use: .modal-right-col * { max-width: 100%; } and avoid min-height on any direct child of right col. (5) SMALL HEIGHT BREAKPOINT: @media (max-height: 800px) { .product-modal-overlay .modal-inner { max-height: 95vh; } .modal-left-col { max-height: calc(95vh - 48px); } .modal-right-col { height: calc(95vh - 48px); } .modal-left-col img { max-height: calc(95vh - 64px); object-fit: contain; } } — uses more vertical space on short screens. (6) SCROLL HINT: Subtle fade-in "scroll for more ↓" text at the bottom of the right column; auto-hides after user scrolls 50px. (7) MOBILE (≤768px): Single-column full-screen modal; image top (natural height up to 45vh, object-fit contain); all content below; modal container scrolls as a whole (overflow-y: auto on modal container itself for mobile only).

- Skeleton loaders: Animated shimmer (background: linear-gradient(90deg, #f0f0f0 25%, #e0e0e0 50%, #f0f0f0 75%); background-size: 200% 100%; animation: shimmer 1.5s infinite).
- Lightbox: Fixed overlay (rgba(0,0,0,0.85)), close on click-outside or X button, keyboard navigable (Esc to close), smooth open/close transition.
- Trust badge pills: display: inline-flex; border-radius: 24px; padding: 5px 14px; border: 1px solid; font-size: 12px; font-weight: 600; cursor: pointer.

6.5 Animations, Visual Effects & Performance

6.5.1 Splash Screen / Page Loading Animation

- Implement a branded splash/loading screen that displays before the main page content appears. The splash screen must be full-viewport, centred, and display a logo animation or brand-styled loader.

- Show the seller's initials circle (same as hero), business name, and an animated loader. Use the seller's colour theme for the splash screen.
- The splash screen must fade out with a smooth CSS transition (opacity 0 + scale-up or slide-up) once the page content is ready. Minimum display time: 1.2 seconds. Maximum: 3 seconds.
- Use CSS loaders from https://freefrontend.com/css-loaders/ for loader style reference. The overlay must be z-index: 9999 and removed from DOM after dismissal.

6.5.2 Parallax Scrolling Effects

- Apply parallax scrolling to the Hero section background. The background should move at a slower rate than the scroll (CSS background-attachment: fixed or GSAP ScrollTrigger).
- Apply subtle depth/parallax to decorative background layers between sections.
- Parallax must be disabled / reduced for prefers-reduced-motion. On mobile, use static background instead.

6.5.3 Scroll-Linked Animations — AOS Integration

Replace manual GSAP/IntersectionObserver scroll animations with AOS (Animate On Scroll) library for all section and element entrance animations(reference implementation in section 9.1).

AOS data attributes by section:

Section / Element	data-aos	data-aos-delay
Hero content (logo, name, tagline)	"fade-up"	0ms, 100ms, 200ms stagger
About section	"fade-right"	0ms
Product cards (each)	"fade-up"	(index % 4) * 100 ms
Category tiles (each)	"zoom-in"	index * 80 ms
Gallery tiles	"fade-up"	index * 50 ms
Social post cards	"slide-up"	index * 100 ms
Review cards	"fade-in"	index * 80 ms
Footer	"fade-up"	0ms

6.5.4 Lazy Loading

- All img tags: loading="lazy" attribute.
- Media gallery: initial 8–10 images eagerly; rest via IntersectionObserver (see Section 5.8.1).
- Instagram embed.js: loaded via IntersectionObserver when Social Posts section enters viewport.
- Social post thumbnails: loading="lazy" + onerror to hide if CDN URL has expired.

6.5.5 Moving / Ambient Animations

- Hero section: add a subtle continuous animation to the background (slow Ken Burns zoom, gradient shift, or floating particle effect using Three.js canvas overlay). This must not cover text or CTAs.

- Trust badges in hero: continuous slow horizontal marquee/ticker animation if more badges than fit on one line.
- All hover effects on interactive elements (buttons, cards, nav links, badges, social icons) must have smooth CSS transitions (transition: all 0.25s ease). No abrupt colour/size jumps.
- All animations must use will-change: transform or opacity where appropriate for GPU compositing. Avoid animating layout properties.

6.5.6 End-to-End Test Checklist

1. Scroll all sections on desktop (≥1280px), tablet (768px), mobile (375px), XS mobile (320px). No layout breaks.
2. Product Catalog grid: 4 cols ≥1280px; correct cols at all breakpoints. "Show More" button appears if >15 products.
3. Category tiles: glassmorphism renders correctly; text readable; AOS zoom-in fires on scroll.
4. Sidebar + main: side-by-side at ≥1024px — never stacked.
5. All trust badges clickable. No seller_id → marketplace URL construction. No external platform URLs.
6. Social icons in Contact and Footer use social_profiles[].url. No mislabeled company_profile fields.
7. Product card click → detail modal with correct JSON data.
8. Category filter pills filter product grid correctly; 10–15 limit applied to filtered results.
9. Gallery: 8–10 images shown initially; lazy-loading fires as user scrolls; social thumbs appear first.
10. Instagram posts: uniform 550px container; placeholder card shows caption/likes/timestamp from JSON.
11. YouTube videos: thumbnail/views/caption on card; iframe embed in modal on click.
12. Hero mobile (≤768px): NO background image; gradient only; text in dark colour on light background.
13. Hero desktop (>768px): banner from YouTube or Facebook social_profiles; overlay rgba(0,0,0,0.85) applied.
14. Reviews: aggregate rating prominent; coloured star icons visible.
15. AOS animations fire on scroll for product cards, category tiles, gallery tiles, social cards.
16. No marketplace-brand names or marketplace URLs in any rendered element.
17. Splash screen appears, ≥1.2s, fades out cleanly.
18. 320px viewport: single column, no overflow, no horizontal scroll, all tap targets ≥44×44px.

7. Business Rules & Conditional Logic

Rule	Condition	Behaviour
Dynamic Section Rendering	Any section data null/empty in JSON	Not rendered. No empty containers.
Data Source Exclusivity	All data	ALL data from provided Catalog JSON. No scraping, crawling, or inference.

Product Data Priority	catalog_items[] vs products[]	Prefer catalog_items[] (enriched). Fall back to products[] if empty.
Category Derivation	product_category values	Derive unique values from catalog_items[]. Count per category. Filter business-type labels.
Business-Type Filter	product_category	Exclude: Manufacturer, Exporter, Trader, Digital creator, Export House, Leader, Importer, Dealer, Wholesaler
Price Display	price fields	price_on_request=true → "Price on Request". is_price_range → range. Else price_single. Append currency+unit.
Image onerror	Any img tag	URL fails → hide silently. Never show broken image icon or empty card.
Product Card Limit	INITIAL_LIMIT = 12 (10–15)	Show only INITIAL_LIMIT cards on load. Rest hidden. Toggle button reveals all.
Gallery Priority	Image ordering	Social thumbnails → media_assets → catalog photos. Deduplicate. First 8–10 shown eagerly; rest lazy-loaded.
Hero Overlay	Always applied	Desktop: rgba(0,0,0,0.85) gradient. Mobile: no overlay (gradient bg, dark text).
Hero Mobile Image	max-width: 768px	Hide background-image. Use var(--color-bg-hero-mob) CSS gradient only.
Social icons source	Contact & Footer icons	Use social_profiles[].url. NOT social_urls[] or company_profile individual social fields.
Social Posts tab visibility	social_profiles[].posts[]	Tab shown ONLY if posts[].length > 0.
Banner priority	Hero background	social_profiles[youtube].banner_url → social_profiles[facebook].banner_url → CSS gradient
Profile pic priority	Hero avatar/footer	social_profiles[instagram] → youtube → facebook → initials circle
City field	company_profile.city	Direct field. No parsing from address.
Description priority	About section	company_profile.description → yt bio → ig bio → fb bio → hidden
CTA Priority	Contact CTAs	WhatsApp primary. Phone same number. Email = emails[0]. All from company_profile.
Google Maps embed	google_location	Iframe embed if non-null. Text-only if null.
Reviews	reviews_summary	Aggregate card if no_of_ratings > 0. Individual cards only if reviews[] non-empty.
No content fabrication	All fields	Absent or null JSON field → hide element. Never fill with guessed content.

No marketplace links	ALL CTAs, badges, links	Zero marketplace platform URLs anywhere on the page. All links from seller's own JSON data.

8. JSON Data Model & Extraction Guide

ALL data on the page must come exclusively from this JSON. No external sources.

8.1 Top-Level JSON Keys

Key	Type	What It Contains
seller_id	string / number	Unique seller identifier (from original data source). Do NOT use to construct any marketplace URL. Show as internal reference only if needed.
company_profile	object	All company-level data — see Section 8.2
products[]	array	Product objects. Fallback if catalog_items empty.
media_assets[]	array	Vision-confirmed media with fingerprint, url, visual_features, visual_description, media_alt_text
catalog_items[]	array	Enriched product objects — prefer over products[]
reviews[]	array	Individual review objects. May be empty.
reviews_summary	object	Aggregate: total_rating, no_of_ratings, rating_comments[]
social_urls[]	array	Root-level social URL strings. Use only as fallback if social_profiles absent.
social_profiles[]	array	PRIMARY social source — per-platform: platform, url, profile_pic_url, banner_url, bio, followers_count, posts[]

8.2 company_profile Object Fields

company_profile.name	Seller/company name. Hero, nav, footer, browser title.
company_profile.address	Full address string. Contact, About, footer.
company_profile.city	Direct city string. Hero badge, About. No parsing needed.
company_profile.emails[]	Array of email strings. Use [0] as primary.
company_profile.phones[]	Array of phone strings. Use [0] for WhatsApp and Call CTAs.
company_profile.website	Website URL. Hero CTA, About, Contact. Null if unavailable.
company_profile.business_type	Business category (e.g. "Jewelry store"). Hero badge and About.
company_profile.description	Seller description (may be null). Primary About text if non-null.
company_profile.rating_value	Numeric rating. Hero badge and Reviews.
company_profile.rating_count	Number of ratings. Used alongside rating_value.

company_profile.google_location	Google Maps URL. About/Contact iframe embed.
company_profile.social_urls[]	Legacy URL array. Use only as final fallback.
company_profile.confidence	INTERNAL — do NOT display.
company_profile.facebook / .instagram / .youtube / .linkedin / .twitter	Individual social URL fields. WARNING: may be mislabeled. ALWAYS prefer social_profiles[].
company_profile.posts_api_url / photos_api_url / reviews_api_url / _raw_text	INTERNAL — do NOT display.

8.3 social_profiles[] Object Fields

Field	Type	Usage
platform	string	"facebook","instagram","youtube","linkedin","twitter"
url	string	Canonical profile URL. Social icons in Contact, Footer, About.
profile_pic_url	string or null	Hero avatar (IG preferred), footer logo, platform header card.
banner_url	string or null	Hero background (YouTube preferred → Facebook). onerror → next source.
bio	string or null	About section description (if company_profile.description null); Social Posts header.
followers_count	number or null	Format: 354K, 1.2M. Hero trust badge (highest), About cards, Social Posts header.
posts[]	array	Post objects. Only show Social tab if posts[].length > 0.
following_count	number or null	Instagram only. Display in Social Posts header if non-null.
posts_count	number or null	Instagram only. Display in Social Posts header if non-null.
_note	string/undefined	INTERNAL — do NOT display.

8.4 social_profiles[].posts[] Object Fields

Field	Type	Usage
type	string	"post" or "video". Determines embed strategy.
post_url	string	IG: blockquote embed permalink. YT: extract video ID for embed URL.
thumbnail_url	string	IG: placeholder card bg (may expire — onerror). YT: stable ytimg.com URL.
caption	string	Truncate to 120 chars on card. Full text in modal.
likes	number	Show on card. May be 0.

comments	number	Show on card. May be 0.
views	number or null	Show only if non-null. Always present for YouTube.
posted_at	string ISO 8601	Format as relative time. Sort descending.

8.5 catalog_items[] Object Fields

Field	Type	Usage
clean_name	string	Product display name on card and modal
fingerprint	string	Unique product ID — matches media_assets[]
product_category	string	Category — filter pills and category tiles
product_sub_type	string	Sub-type — secondary chip on card
buyer_persona	string	"Ideal for: ..." in modal
description	string	Full product description
tags[]	array	First 3 on card; all in modal
alt_text	string	img alt attribute
specifications{}	object	Key-value specs — exclude null values in modal
b2b_attributes{}	object	moq, price_unit, hsn_code, gst_percent, country_of_origin, brand
price_single/min/max	number or null	Use with is_price_range and price_on_request
price_on_request	boolean	If true → "Price on Request"
is_price_range	boolean	If true → show price_min–price_max
price_unit / currency	string	Append to price display
primary_photo_url	string or null	Card image — onerror hides
photo_urls[]	array	Modal image carousel
source_url	string or null	"View Original Listing" in modal if non-null
in_stock	boolean	"In Stock" green / "Out of Stock" grey
used_vision	boolean	If true → look up media_assets[fingerprint]
media_url	string	Fallback if primary_photo_url null

8.6 media_assets[] Object Fields

fingerprint	Matches catalog_items[N].fingerprint — use to pair media with product
url	Direct image URL — use as img src with onerror handler
scene_type	Media scene type (e.g. "product_image") — use for gallery filtering

media_confidence	Confidence score (0–1) — only display if ≥ 0.7
visual_features{}	Object with shape_visible, color_appearance, clarity_appearance, size_impression, presentation — show as additional spec rows in modal
visual_description	Full sentence image description — used as gallery/lightbox caption
media_alt_text	Accessibility alt text — applied to img alt attribute
used_vision	boolean — whether AI vision was used to classify this asset
classified_at	ISO 8601 timestamp — INTERNAL; do NOT display

8.7 reviews_summary Object Fields

reviews_summary.total_rating	Overall star rating (e.g. 3.3) — displayed in Reviews aggregate card
reviews_summary.no_of_ratings	Count of ratings (e.g. 23) — displayed as "Based on N ratings"
reviews_summary.rating_comments[]	Array of review comment strings — if non-empty, display as individual comment cards

8.8 Data Extraction Reference Implementation**(reference)**

```
function buildCatalogPage(json) {
  // ── IDENTITY ──────────────────────────────────────────────
  const seller_id      = json.seller_id;   // internal ID only — NO marketplace URL
  const seller_name    = json.company_profile.name;
  const business_type  = json.company_profile.business_type;
  const address        = json.company_profile.address;
  const city           = json.company_profile.city;
  const website        = json.company_profile.website;
  const google_map     = json.company_profile.google_location;
  const description    = json.company_profile.description;

  // ── CONTACT ───────────────────────────────────────────────
  const phone_primary  = json.company_profile.phones?.[0] || null;
  const email_primary  = json.company_profile.emails?.[0] || null;

  // ── SOCIAL PROFILES ───────────────────────────────────────
  const social_profiles = json.social_profiles || [];
  const getSP = (p) => social_profiles.find(s => s.platform === p) || null;
  const ig_profile = getSP("instagram");
  const yt_profile = getSP("youtube");
  const fb_profile = getSP("facebook");

  const hero_banner_url  = yt_profile?.banner_url || fb_profile?.banner_url || null;
  const seller_logo_url  = ig_profile?.profile_pic_url || yt_profile?.profile_pic_url
                        || fb_profile?.profile_pic_url || null;
  const page_description = description || yt_profile?.bio || ig_profile?.bio ||
fb_profile?.bio || "";

  const social_icons = social_profiles.filter(s=>s.url)
```

```
    .map(s=>({platform:s.platform, url:s.url, pic:s.profile_pic_url}));

  const platforms_with_posts = social_profiles
    .filter(s => s.posts?.length > 0)
    .map(s=>({...s, posts:[...s.posts].sort((a,b)=>new Date(b.posted_at)-new
Date(a.posted_at)).slice(0,5)}));

  const max_followers = Math.max(...social_profiles.map(s=>s.followers_count||0));

  // ── PRODUCTS ──────────────────────────────────────────────
  const total_products  = json.products?.length || 0;   // full catalog count
  const items = (json.catalog_items?.length > 0) ? json.catalog_items :
(json.products||[]);
  const showcased_count = items.length;
  const product_count_label = showcased_count < total_products
    ? `Showing ${showcased_count} of ${total_products} Products`
    : `${total_products} Products`;

  // ── CATEGORIES ────────────────────────────────────────────
  const EXCLUDE = ["Manufacturer","Exporter","Trader","Digital creator",
    "Export House","Leader","Importer","Dealer","Wholesaler"];
  const categories = [...new
Set(items.map(i=>i.product_category).filter(c=>c&&!EXCLUDE.includes(c)))];

  // ── GALLERY — social thumbs FIRST ─────────────────────────
  const social_thumbs = social_profiles.flatMap(sp=>(sp.posts||[])

.filter(p=>p.thumbnail_url).map(p=>({url:p.thumbnail_url,badge:"Social",alt:p.caption?.sl
ice(0,80)||""})));
  const media_imgs = (json.media_assets||[]).filter(m=>m.media_confidence>=0.8)

.map(m=>({url:m.url,badge:"Product",alt:m.media_alt_text,caption:m.visual_description}));
  const catalog_imgs =
items.flatMap(i=>(i.photo_urls||[]).map(u=>({url:u,badge:"Gallery",alt:i.alt_text||""})))
;
  const gallery_images = [...social_thumbs,...media_imgs,...catalog_imgs]
    .filter((v,i,a)=>a.findIndex(x=>x.url===v.url)===i);

  // ── REVIEWS ───────────────────────────────────────────────
  const rating_value    = json.reviews_summary?.total_rating ||
json.company_profile?.rating_value || null;
  const rating_count    = json.reviews_summary?.no_of_ratings ||
json.company_profile?.rating_count || null;
  const review_comments = []; // CHANGE 6: DO NOT use rating_comments[] — use reviews[]
only to avoid duplication
  const reviews         = json.reviews || [];

  return {
    seller_id, seller_name, business_type, address, city, website, google_map,
    description, page_description, hero_banner_url, seller_logo_url,
    social_icons, platforms_with_posts, max_followers, social_profiles,
    phone_primary, email_primary, items, total_products, showcased_count,
product_count_label,
    categories, gallery_images, rating_value, rating_count, review_comments, reviews
  };
}
```

8.9 Matching Media Assets to Products

Use the fingerprint field to match media_assets[] entries to their corresponding catalog_items[] entry:

```
function getMediaForItem(item, media_map) {
  return media_map[item.fingerprint] || null;
}
// In product detail modal:
const matched_media = getMediaForItem(item, media_map);
if (item.used_vision && matched_media) {
  // Append visual_description to product description
  // Show visual_features as additional specification rows
  // Use media_alt_text as img alt attribute
}
```

8.10 Social URL Platform Detection (Legacy Fallback)

Use this function ONLY as a fallback when social_profiles[] is absent or when processing raw social_urls[] entries. Always prefer social_profiles[].url over this approach.

```
function detectPlatform(url) {
  if (!url) return null;
  if (url.includes("instagram.com"))  return "instagram";
  if (url.includes("facebook.com"))   return "facebook";
  if (url.includes("youtube.com") || url.includes("youtu.be")) return "youtube";
  if (url.includes("linkedin.com"))   return "linkedin";
  if (url.includes("twitter.com") || url.includes("x.com")) return "twitter";
  if (url.includes("api.whatsapp.com") || url.includes("wa.me")) return "whatsapp";
  if (url.includes("pinterest.com"))  return "pinterest";
  if (url.includes("tiktok.com"))     return "tiktok";
  return null;
}
```

9. Technical Implementation Guide

9.1 AOS (Animate On Scroll) Library Integration

AOS setup (add to HTML entry point):

```
<!-- In <head> -->
<link rel="stylesheet" href="https://unpkg.com/aos@2.3.4/dist/aos.css" />

<!-- Before </body> -->
<script src="https://unpkg.com/aos@2.3.4/dist/aos.js"></script>
<script>
  document.addEventListener("DOMContentLoaded", () => {
    AOS.init({
      offset:   100,
      duration: 800,
      easing:   "ease-in-out",
      once:     true,
    });
  });
</script>

/* React/Next.js integration */
import AOS from "aos";
import "aos/dist/aos.css";
```

```
useEffect(() => { AOS.init({offset:100, duration:800, easing:"ease-in-out", once:true});
}, []);
// After dynamic content is added:
AOS.refresh();  // re-scan DOM for new [data-aos] elements
```

9.2 Product Card Limit — Show More Toggle

HTML structure:

```
<section id="products" data-aos="fade-up">
  <div class="product-grid" id="product-grid">
    <!-- Cards injected by JS — first INITIAL_LIMIT visible, rest hidden -->
  </div>
  <div class="show-more-wrap">
    <button id="show-more-btn" class="btn-show-more"
            style="display:none">  <!-- hidden if total ≤ INITIAL_LIMIT -->
      Show <span id="hidden-count">0</span> More Products
    </button>
  </div>
</section>

/* CSS for hidden cards */
.product-card.hidden-product { display: none; }
```

9.3 Image Gallery Lazy Loading

Sentinel-based lazy loading:

```
<div class="gallery-grid" id="gallery-grid">
  <!-- First GALLERY_INITIAL images rendered eagerly -->
</div>
<div class="gallery-sentinel"></div>  <!-- triggers lazy load -->

/* IntersectionObserver */
const io = new IntersectionObserver(([e]) => {
  if (!e.isIntersecting || !lazyQueue.length) return;
  const batch = lazyQueue.splice(0, 6);
  batch.forEach(img => {
    const tile = createGalleryTile(img);
    tile.setAttribute("data-aos", "fade-up");
    document.getElementById("gallery-grid").appendChild(tile);
  });
  AOS.refresh();
  if (!lazyQueue.length) io.disconnect();
}, { rootMargin: "200px" });
io.observe(document.querySelector(".gallery-sentinel"));
```

9.4 Mobile Responsive Rules (320px–768px)

```
/* ——— BREAKPOINT: 768px and below ———————————————————— */
@media (max-width: 768px) {
  /* Change #7: Remove hero banner image on mobile */
  .hero-banner {
    background-image: none !important;
    background: linear-gradient(135deg, #F5F1E8 0%, #EDE8DC 50%, #F5F1E8 100%);
  }
  .hero-overlay { background: transparent; }
```

```
  .hero-name, .hero-tagline, .hero-meta { color: var(--color-text-primary); }

  /* Contact sidebar moves below main content */
  .page-layout { flex-direction: column; }
  .contact-sidebar { position: static; width: 100%; }

  /* Hamburger nav */
  .hamburger { display: flex; }
  .desktop-nav { display: none; }
}

/* ─── BREAKPOINT: 480px and below ──────────────────────────── */
@media (max-width: 480px) {
  /* Product grid: single column */
  .product-grid { grid-template-columns: 1fr; gap: 12px; padding: 0 8px; }

  /* Category tiles: 2 columns max */
  .category-grid { grid-template-columns: repeat(2, 1fr); gap: 8px; }

  /* Gallery: single column */
  .gallery-grid { grid-template-columns: 1fr; }

  /* Hero: tighter spacing */
  .hero-content { padding: 24px 16px; }
  .hero-name { font-size: clamp(20px, 7vw, 28px); }

  /* Trust badge strip: horizontal scroll */
  .badge-strip { overflow-x: auto; flex-wrap: nowrap; -webkit-overflow-scrolling: touch;
}

  /* Filter pills: horizontal scroll */
  .filter-pills { overflow-x: auto; flex-wrap: nowrap; padding-bottom: 8px; }

  /* Tabs: wrap or scroll */
  .platform-tabs { overflow-x: auto; }
}

/* ─── BREAKPOINT: 360px and below (XS) ─────────────────────── */
@media (max-width: 360px) {
  /* Single column category tiles */
  .category-grid { grid-template-columns: 1fr; }

  /* Minimum font sizes */
  body { font-size: 13px; }
  .hero-name { font-size: 20px; }
  .product-card { padding: 12px; }

  /* Buttons: full width */
  .hero-ctas { flex-direction: column; width: 100%; }
  .hero-ctas .btn { width: 100%; text-align: center; }

  /* Touch targets: 44px minimum */
  .btn, .nav-link, .filter-pill, .badge, .hamburger, .social-icon {
    min-height: 44px;
    min-width: 44px;
    display: flex;
    align-items: center;
    justify-content: center;
```

```
  }
}
```

9.5 Glassmorphism Utility Class

```
/* Glassmorphism utility — apply to category tiles, modal backgrounds */
.glass {
  background: rgba(255, 255, 255, 0.15);
  backdrop-filter: blur(10px);
  -webkit-backdrop-filter: blur(10px);
  border: 1px solid rgba(255, 255, 255, 0.25);
  border-radius: 16px;
  box-shadow: 0 4px 24px rgba(0,0,0,0.08);
}
.glass-warm { background: rgba(245,241,232, 0.55); }  /* warm cream tint */
.glass-beige { background: rgba(237,232,220, 0.55); } /* warm beige tint */
```

9.6 Instagram Embed Sizing Fix

```
/* Uniform Instagram embed container — Change #10 */
.ig-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(320px,1fr));
gap: 20px; }

.ig-post-wrapper {
  width: 100%;
  max-width: 400px;
  height: 550px;
  overflow: hidden;
  border-radius: 12px;
  position: relative;
  background: var(--color-bg-section);
}

.ig-post-wrapper .instagram-media {
  transform: scale(0.95);
  transform-origin: top center;
  width: 100% !important;
  min-width: unset !important;
}

/* Placeholder card: shown until embed.js loads */
.ig-placeholder {
  position: absolute; inset: 0;
  background-size: cover; background-position: center;
  display: flex; flex-direction: column; justify-content: flex-end;
  padding: 16px; border-radius: 12px;
  background-color: var(--color-bg-card);
}
.ig-placeholder-info { background: rgba(0,0,0,0.5); border-radius: 8px;
  padding: 8px 12px; color: #fff; font-size: 13px; }

/* Timeout fallback: if embed.js does not load within 5s, keep placeholder */
setTimeout(() => {
  document.querySelectorAll(".ig-placeholder")
    .forEach(el => { if (!el.dataset.loaded) el.style.display = "flex"; });
}, 5000);
```

10. Exceptions & Edge Cases

Scenario	Handling	JSON Check
No marketplace URL needed	seller_id is internal only. No URL is constructed from it.	All CTAs — none use seller_id for URL construction
social_profiles[] absent	Social Posts hidden; hero gradient; initials circle; social_urls[] fallback	!json.social_profiles || len === 0
social_profiles[p].posts[] empty	NO tab for that platform. No skeleton. Hide entirely.	posts.length === 0
Banner URLs both null	CSS gradient fallback applied. Never leave banner empty.	yt.banner_url===null && fb.banner_url===null
Mobile viewport ≤768px	Hero background-image removed. Light gradient applied. Text colour switched to dark.	CSS: @media max-width: 768px
Total products > 15	"Show More" button shown with hidden count. Button hidden if products ≤ INITIAL_LIMIT.	allItems.length > INITIAL_LIMIT
Instagram thumbnail expires (403)	onerror hides thumbnail; placeholder still shows JSON text data (caption, likes, timestamp)	onerror on thumbnail_url img
Instagram embed.js fails	After 5s timeout, keep placeholder permanently. Never show broken embed.	setTimeout 5000ms check
company_profile.description null	Fall through: yt bio → ig bio → fb bio. If all null, description hidden.	description === null
No products / catalog_items	Product Catalog + Category sections hidden entirely	both arrays empty
Image URL fails to load	onerror removes tile silently — no broken icon, no empty card	Any img tag
price_on_request true	"Price on Request" shown; all numeric prices hidden	price_on_request === true
specifications{} has null values	Exclude null rows from modal table	Object.entries(spec).filter(([k,v])=>v!==null)

seller_id is number	Coerce: String(json.seller_id) — for internal display only	typeof seller_id !== "string"
Spline scene unavailable	Fall back to Three.js canvas; if not feasible, CSS-only animation	n/a
320px viewport	Single column everything; font-size reduced; category tiles 1-col; all tap targets ≥44px.	@media max-width:360px

11. Acceptance Criteria

11.1 Generalisation

- Page works correctly with any valid seller Catalog JSON.
- No seller-specific value hard-coded anywhere in HTML/CSS/JS.
- All data exclusively from the provided JSON — no scraping, crawling, or fabrication.
- Zero instances of marketplace platform names, logos, or URLs in any rendered element. Verified by document text search.

11.2 Functional

- No breadcrumb anywhere.
- All trust badges clickable. No badge constructs a marketplace URL.
- Product catalog shows cards with name, image, price, category, in-stock, description — all from JSON.
- Product grid: 10–15 cards on initial load. "Show More" button if total > INITIAL_LIMIT.
- Product detail modal shows full specs, description, images, B2B attributes, buyer persona — from JSON.
- Category filter pills work; 10–15 limit applied per category.
- Category tiles: glassmorphism renders; AOS zoom-in on scroll; text readable.
- Gallery: social thumbnails appear first; product images second. 8–10 shown initially; rest lazy-loaded.
- Social Posts: only tabs with posts[].length > 0 shown.
- Instagram embed container: uniform 550px height; placeholder card from JSON fields; no broken embeds.
- YouTube videos: thumbnail card, iframe embed in modal.
- All CTAs limited to: WhatsApp, phone, email, seller website, social profiles — no marketplace links.
- Social icons: social_profiles[].url (not company_profile individual fields).
- Google Maps iframe if google_location non-null; text-only if null.

11.3 Responsive Design

- 320px (XS mobile): single column, no overflow, all tap targets ≥44px.

- 360–479px: 1-col products, 1-col gallery, reduced fonts.
- 480–767px: 1-col products, 2-col gallery, sticky CTA bar, hamburger nav.
- 768–1023px: 2-col products, no sidebar, mobile CTA bar.
- 1024–1279px: 3-col products, 290px sidebar.
- ≥1280px: 4-col products, 320px sidebar, 3-col gallery.
- Hamburger: hidden at hero transparent state; shown only in sticky opaque nav. All menu links work.

11.4 Animations & Visual Polish

- Splash screen: appears on load, ≥1.2s, fades cleanly.
- Hero desktop: banner image from social_profiles (YouTube or Facebook); overlay rgba(0,0,0,0.85).
- Hero mobile (≤768px): NO background image; CSS gradient; dark text on light background.
- AOS animations fire on scroll for: product cards (fade-up), category tiles (zoom-in), gallery tiles (fade-up), social cards (slide-up), review cards (fade-in), section headers.
- Light-white colour palette applied throughout — NOT plain #FFFFFF everywhere.
- CSS variables (--color-*) define all colours. No hardcoded hex in component CSS.
- All button text legible — no white-on-white at any state.
- Spline or Three.js ambient effect in hero section.

A floating “Back to Top” button must be present globally on the page. Specification: (1) Position: fixed, bottom-right corner, with margins: bottom: 28px; right: 28px (on mobile: bottom: 80px to stay above the sticky CTA bar; right: 16px). (2) Visibility: the button is hidden (opacity: 0, pointer-events: none) when the user is at or near the top of the page (scrollY < 300px). It fades in (opacity: 1, transition: 0.3s ease) once the user has scrolled more than 300px down the page. (3) Appearance: circular button, 48x48px, background: var(--color-accent) or a semi-transparent dark background (rgba(0,0,0,0.55)), white upward chevron or arrow icon centered inside, border-radius: 50%, box-shadow: 0 4px 16px rgba(0,0,0,0.18). (4) Motion / floating effect: apply a gentle vertical float animation — @keyframes float { 0%,100% { transform: translateY(0); } 50% { transform: translateY(-6px); } } with animation: float 2.8s ease-in-out infinite. Additionally apply a subtle cloud-like pulsing glow: box-shadow alternates between 0 4px 16px rgba(0,0,0,0.18) and 0 8px 28px rgba(var(--color-accent-rgb), 0.35) on the same keyframe cycle. (5) On click: smooth scroll to top (window.scrollTo({ top: 0, behavior: ‘smooth’ })). (6) Desktop only for the floating animation; on mobile the button is present but the float animation is disabled (reduced motion / performance). (7) z-index: 999 — must appear above all sections including sticky nav and sticky CTA bar.

SPLASH CURSOR (ReactBits): Replace the previous circular cursor follower implementation with the Splash Cursor animation from ReactBits (https://www.reactbits.dev/animations/splash-cursor). Implementation details: (1) Use the SplashCursor React component exactly as provided by ReactBits — import and drop into the page layout so it renders globally. The component attaches to the page root and tracks the mouse independently. (2) COLOR: Configure as multi-color rainbow mode. Use a cycling hue rotation across the splash trail — cycling through red, orange, yellow, green, cyan, blue, violet in sequence. Set the splash/fluid color to cycle with hue shifts using HSL: hsl(calc(var(--hue, 0) + N * 30deg), 80%, 55%) pattern, or use the ReactBits color configuration options if available. (3) SIZE: Make the splash effect noticeably smaller than the default — reduce the splash radius / canvas scale to approximately 60-70% of the default. The effect should be subtle and compact, not large or dominating. (4) GLOW: Reduce glow/bloom intensity. Set any glow or blur radius to a low value (e.g. blur: 8px instead of default). The effect should be present but understated — less fancy, less distracting. (5)

DESKTOP ONLY: Disable completely on viewports <= 1024px and touch devices (@media (pointer: coarse) — hide the canvas element entirely). No splash cursor on mobile or tablet. (6) The splash cursor must NOT replace or interfere with the native OS cursor. The native pointer/cursor must remain visible at all times. (7) Z-index: the splash canvas sits at z-index 9998, below any modal or dropdown (z-index 9999+). (8) The SplashCursor component should be mounted once at app root level (outside individual section components) so it persists across the full page scroll. Reference: https://www.reactbits.dev/animations/splash-cursor

(Comment all code with explanations. Maintain clean indentation.)

Appendix A: Light-White Colour Palette Recommendations

Reference: https://colorhunt.co/palettes/light-white — Three curated palette options below. Select one and apply consistently throughout the page via CSS variables.

Palette Option 1 — Warm Cream & Parchment

Inspired by Gumroad / Squarespace warm editorial aesthetic

--color-bg-primary	#F5F1E8 — Soft cream (main page background)
--color-bg-section	#EDE8DC — Warm parchment beige (alternate sections)
--color-bg-card	#FDFCFA — Near-white with warm tint (card backgrounds)
--color-text-primary	#2C2118 — Deep warm charcoal (headings, body text)
--color-text-secondary	#6B5E52 — Warm medium brown (secondary text)
--color-text-muted	#9C8F85 — Muted warm grey (timestamps, labels)
--color-border	#DDD7CE — Warm light beige border
--color-accent	#5B4FBE — Muted purple accent (CTAs, highlights)
--color-accent-green	#10B981 — Green (WhatsApp, in-stock)
Hero mobile gradient	135deg, #F5F1E8 0%, #EDE8DC 50%, #F5F1E8 100%

Palette Option 2 — Soft Lavender & Ivory

Inspired by Framer / Pitch minimal modern aesthetic

--color-bg-primary	#F7F6FC — Soft lavender-white (main page background)
--color-bg-section	#EEEAF8 — Light lavender (alternate sections)
--color-bg-card	#FFFFFF — Pure white cards with warm shadow
--color-text-primary	#1C1B2E — Deep blue-charcoal
--color-text-secondary	#5E5B78 — Muted purple-grey
--color-text-muted	#9B98B2 — Light purple-grey
--color-border	#DAD6F0 — Soft lavender border
--color-accent	#6366F1 — Indigo accent (CTAs, highlights)
--color-accent-green	#10B981 — Green (WhatsApp, in-stock)

Hero mobile gradient	135deg, #F7F6FC 0%, #EEEAF8 50%, #F7F6FC 100%

Palette Option 3 — Minimal Warm Grey & Cloud White

Inspired by Linear / Notion clean minimal aesthetic

--color-bg-primary	#F9F8F7 — Cloud white (main page background)
--color-bg-section	#F2F0EE — Warm light grey (alternate sections)
--color-bg-card	#FFFFFF — Pure white cards
--color-text-primary	#1A1917 — Near-black warm (headings, body)
--color-text-secondary	#6B6660 — Warm medium grey (secondary)
--color-text-muted	#9E9B97 — Muted warm grey (meta, labels)
--color-border	#E5E2DE — Warm grey border
--color-accent	#2563EB — Bold blue accent (CTAs)
--color-accent-green	#10B981 — Green (WhatsApp, in-stock)
Hero mobile gradient	135deg, #F9F8F7 0%, #F2F0EE 50%, #F9F8F7 100%

Appendix B: 320px Responsive Design Checklist

Use this checklist when testing or building the 320px viewport (iPhone SE, older Android devices, browser devtools 320px):

B.1 Navigation

- Hamburger icon: visible (display:flex), min 44×44px tap target, visibility:visible only when nav is sticky
- Desktop nav links: hidden at ≤768px (display:none)
- Nav menu (open state): full-width overlay, vertical links, 44px min height each, 12px padding between items
- Nav brand/logo: shrinks to 32px max-height; business name truncated with text-overflow:ellipsis

B.2 Hero Section

- Background: CSS gradient only (no image) at ≤768px
- Text: dark colour on light gradient background
- Business name: clamp(20px, 7vw, 26px) — no overflow
- Tagline: clamp(13px, 4vw, 16px)
- CTA buttons: full-width, stacked (flex-direction:column), min 44px height each
- Avatar/logo: 60–72px circle; centred or left-aligned
- Trust badges: horizontal scroll (overflow-x:auto, flex-wrap:nowrap)

B.3 Product Grid

- Single column (grid-template-columns: 1fr) at ≤480px
- Card: full-width with 8–12px horizontal padding

- Card image: aspect-ratio 4/3; object-fit:cover; no horizontal overflow
- "Show More" button: full-width; 44px height; clear visible label
- Filter pills: horizontal scroll; no wrap; -webkit-overflow-scrolling:touch

B.4 Category Tiles

- 360px and below: single column (grid-template-columns: 1fr)
- 361px–480px: two columns (grid-template-columns: repeat(2,1fr))
- Tile text: dark on glassmorphic background; min font-size 13px
- Tile height: auto (do not fix at 320px — allow content to breathe)

B.5 Media Gallery

- Single column at ≤480px
- Images: width:100%; height:auto or fixed aspect-ratio (4/3); no horizontal overflow
- Lightbox: full-screen overlay; close button ≥44×44px; Esc keyboard close

B.6 Social Posts

- Platform tabs: horizontal scroll at ≤480px
- Instagram container: max-width:100%; height:450px at ≤360px
- YouTube cards: full-width; 16/9 aspect-ratio thumbnail
- Social profile header: avatar 48px; bio 2 lines max; follow button 44px min height

B.7 Touch Targets (WCAG 2.5.5)

- ALL interactive elements: minimum 44×44px clickable area
- Nav links, filter pills, badges, social icons, card buttons: enforce with min-height:44px + min-width:44px
- Use padding rather than explicit width/height to preserve visual styling while meeting size requirement

```
/* Touch target enforcement — apply to all interactive elements */
.btn, .nav-link, .filter-pill, .badge, .social-icon,
.hamburger, .category-tile, .gallery-tile {
  min-height: 44px;
  min-width: 44px;
  display: flex;
  align-items: center;
  justify-content: center;
  cursor: pointer;
}
```

B.8 Typography Scaling at 320px

Heading (H1)	clamp(20px, 7vw, 26px)
Heading (H2)	clamp(17px, 5.5vw, 22px)
Body text	clamp(13px, 4vw, 15px)
Product card name	clamp(13px, 4vw, 16px)

Badge / label text	10px minimum — do not scale below this
Price	clamp(15px, 5vw, 18px)